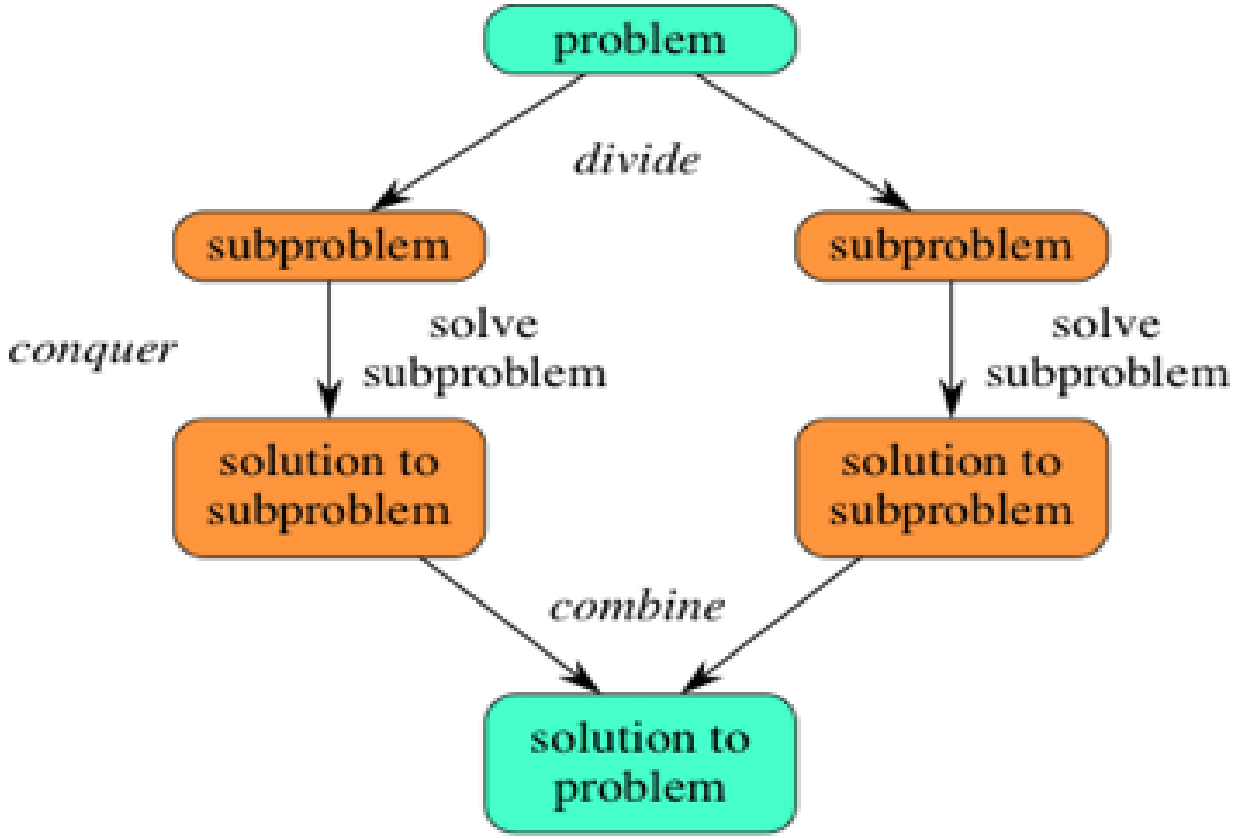
Divide-and-Conquer

- The most-well known algorithm design strategy useful for solving many problems.
- There are three stages in Divide and conquer strategy:

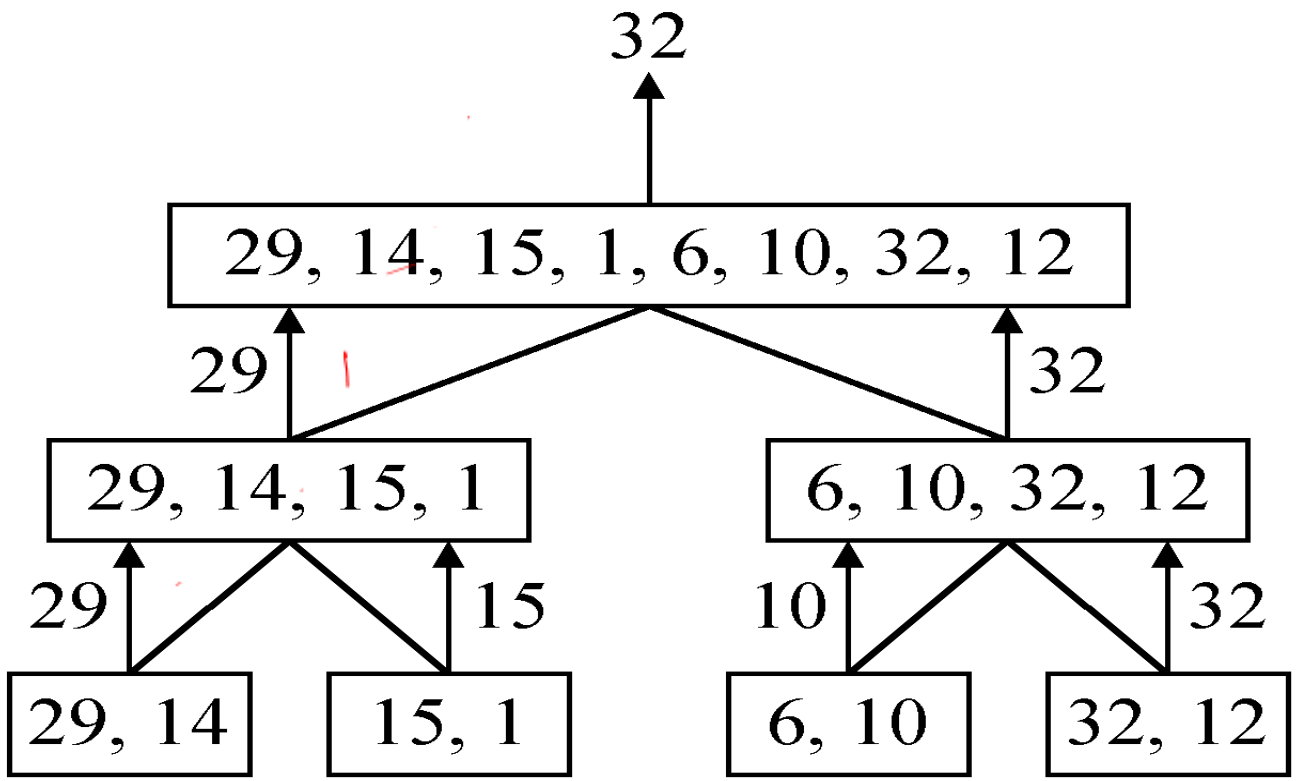
1. DIVIDE: Divide the problem into two or more sub problems.
2. CONQUER: Solve the sub problems recursively or non-recursively.
3. COMBINE: Combine the results of the sub problems to derive a solution of the given problem.

Divide-and-Conquer

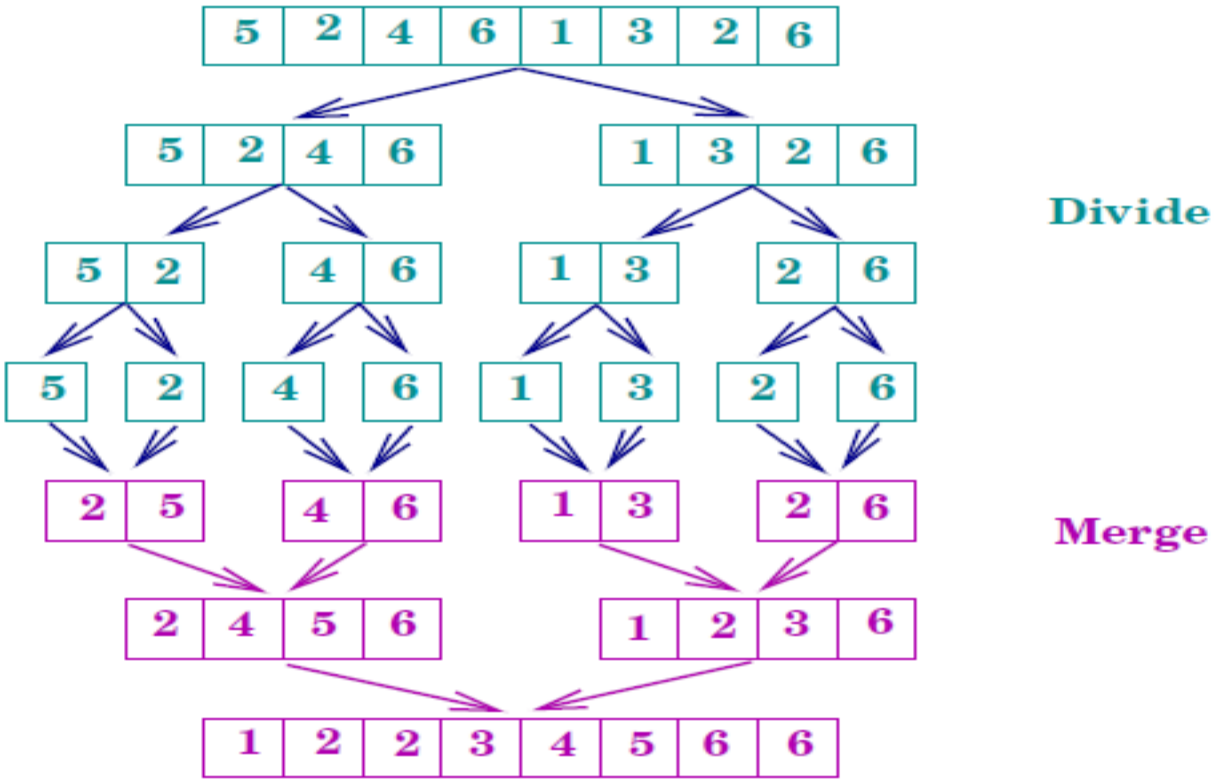


Divide-and-Conquer

- finding the maximum of a set S of n numbers

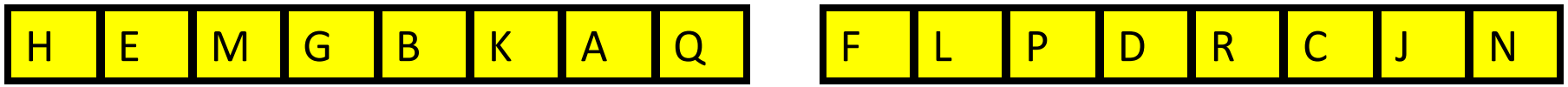


Merge Sort

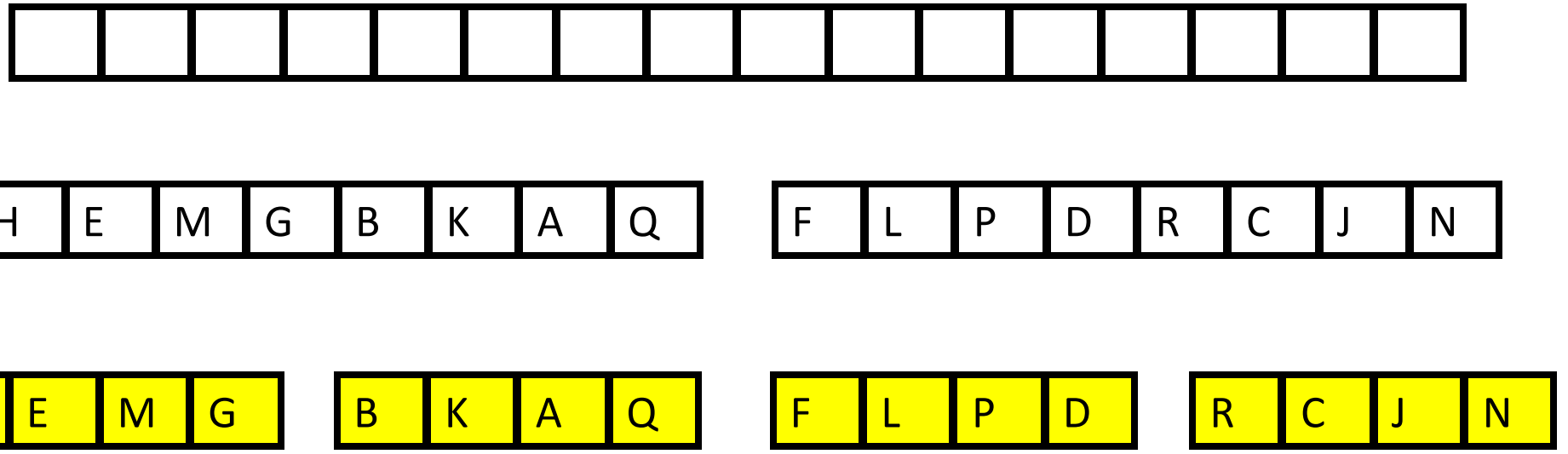

5 2 4 6 1 3 2 6
5 2 4 6
1 3 2 6
5 2
4 6
1 3
2 6
5
2
4
6
1
3
2
6
2 5
4 6
1 3
2 6
2 4 5 6
1 2 3 6
1 2 2 3 4 5 6 6
Divide
Merge

Subdivide the sorting task

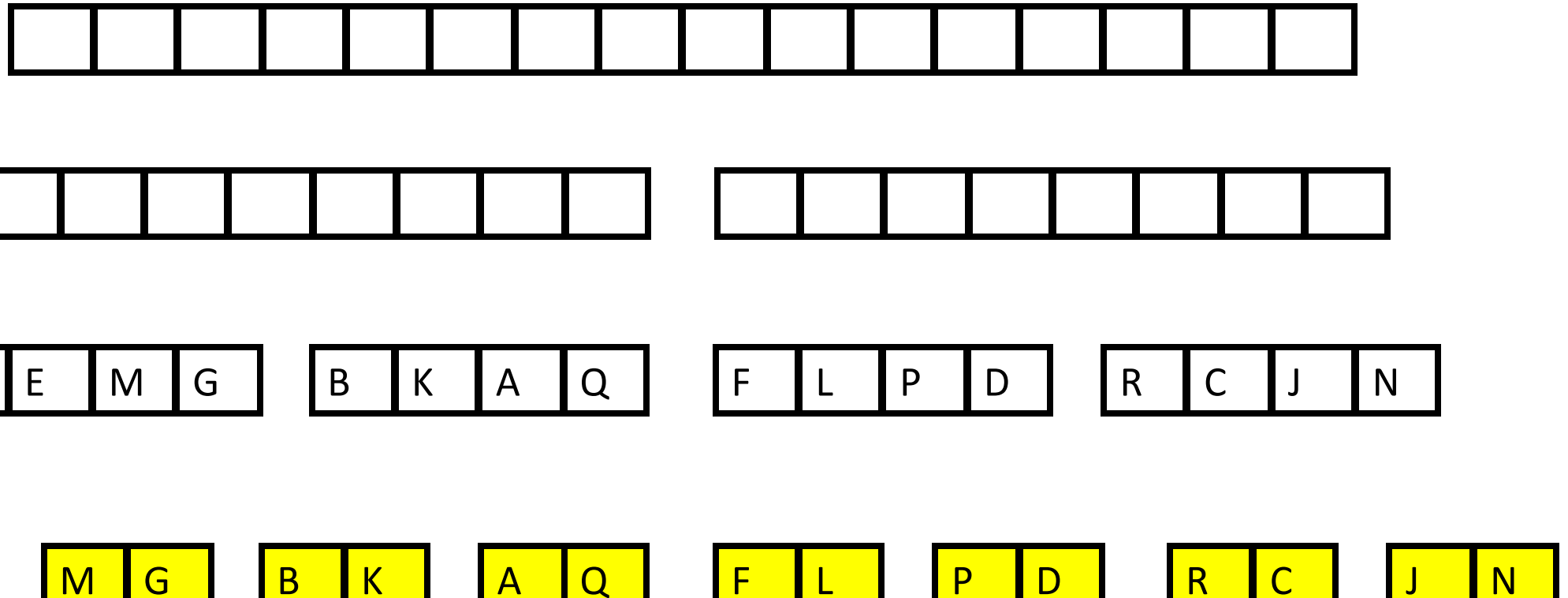




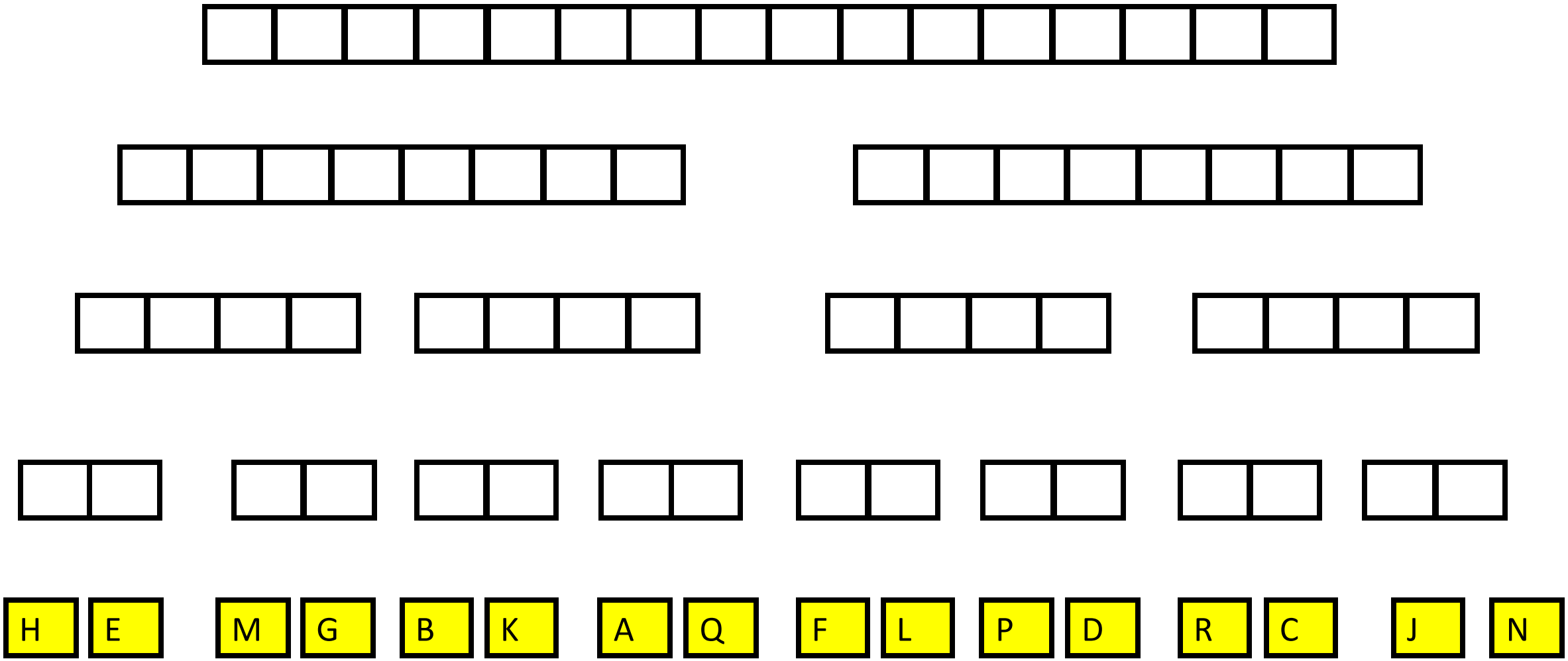
Subdivide again



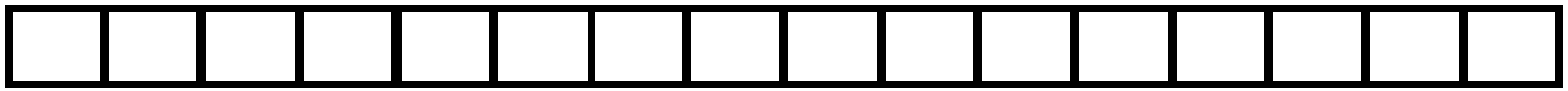
And again

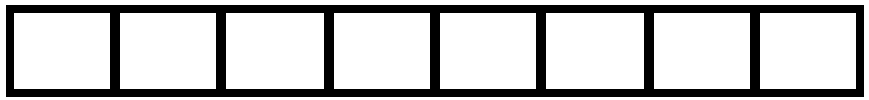


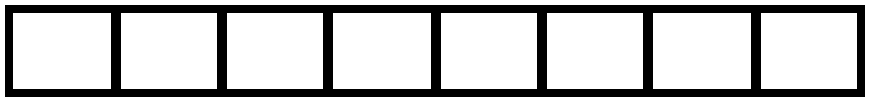
And one last time

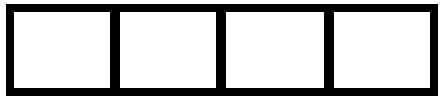


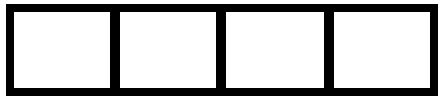
Now merge

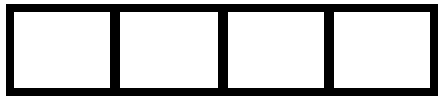


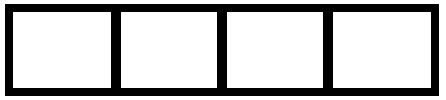


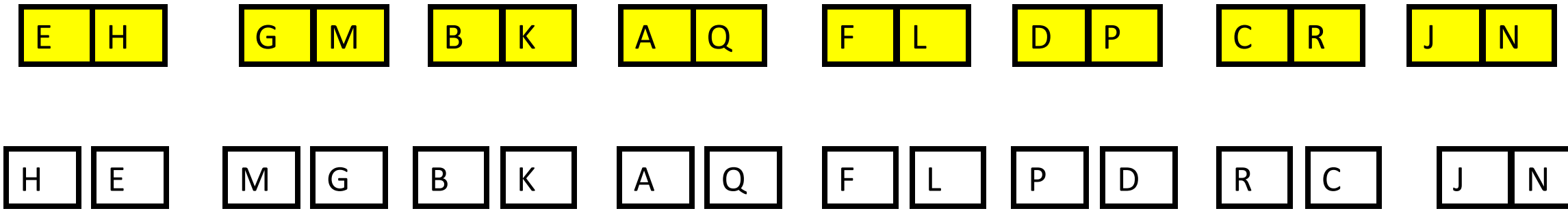




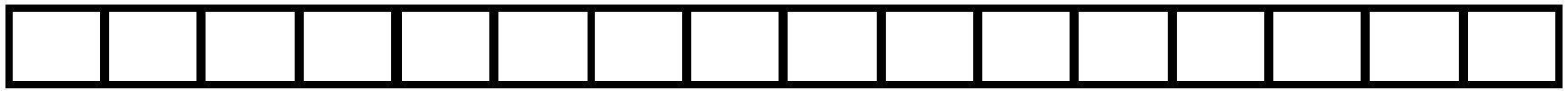


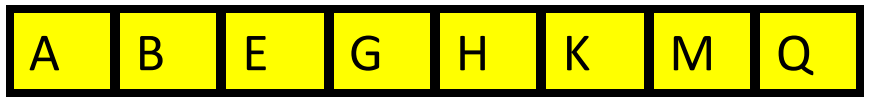


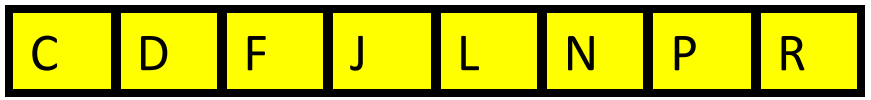




And again







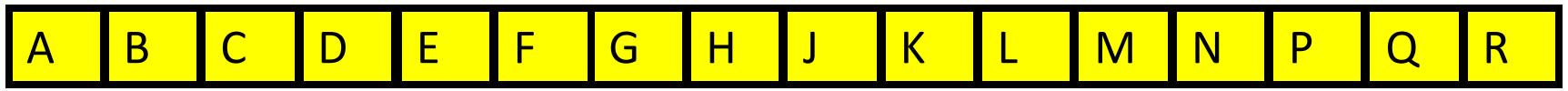








And one last time







Done!



Control abstraction for divide and conquer

```
Algorithm   DandC (P)// P-Problem
  {
      if small (P) then return S(P);
     else
     {
     divide P into k subproblems P1, P2 -, Pk.
      Apply DandC algorithm to each subproblem Pi, l≤i≤k
      return (Combine (DandC(P1),DandC(P2),…      ……..  ,DandC(Pk));
        }
       }
```

Analysis of the Algorithm

- Let us assume that T(n) denotes the time complexity of the problem P and $T(n_i)$ is the Time complexity of the problem P_i, $l \leq i \leq k$. Let g(n) be time required to compute the solution when the problem is small

  Then the time complexity T(n) can be given as follows :

$$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \ldots + T(n_k) + f(n), & \text{otherwise} \end{cases}$$

  Where $T(n_i)$ is the time complexity of the problem P_i and $n_1+n_2 + \ldots + n_k=n$, f(n) be the time required for combining the solutions .

Analysis of the Algorithm

- Time complexity of the divide and conquer algorithms is given by

$$T(n) = \begin{cases} g(n) & n=1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

Where a and b are constants

- Here g(n) is the time required to compute the solution when the problem is small and f(n) is the time required for combining the solutions .

Divide-and-Conquer applications

- Binary search
- Finding the maximum and minimum
- Sorting:Merge sort and Quick sort
- Defective chessboard
- Multiplication of long integers
- Binary tree traversals
- Matrix multiplication: Strassen’s algorithm

Binary Search

Input: A sorted sequence of n elements stored in an array.

Output: The position of x (to be searched).

Step 1: If only one element remains in the array, solve it directly.

Step 2: Compare x with the middle element of the array.

Step 2.1: If x = middle element, then output it and stop.

Step 2.2: If x < middle element, then recursively solve the problem with x and the left half array.

Step 2.3: If x > middle element, then recursively solve the problem with x and the right half array.

Recursive Binary Search Algorithm

```
Algorithm BinSrch(A, i, j, x)
  { // A→ Array, i→ least index, j → highest index, x → element for search.
        if (i = j) then
              { if (x == A[i]) then return i;
                    else return 0;
               }
        else
              {
                    mid ← (i+j)/2;
                   if (x ==A[mid]) then return mid;
                   else if (x < A⌈mid]) then
                         return BinSrch(A, i, mid –1, x)
                     else
                         return BinSrch (A, mid + 1, j, x)
               }
        }
```

```
Algorithm binarySearch( A, low,high ,int x)
   {
   while (low <= high)
   {
            int mid = (low + high) / 2;
             if (x == A[mid])
            { return mid;
            }
            else if (x < A[mid])
            {
            high = mid - 1;
            } else
            { low = mid + 1; }
             }
     }
            return -1;
   }
```

Binary Search algorithm Analysis

- In recursive binary search we have

  $T(n) = 1 + T(n/2)$
  $T(1) = 1$

- $T(n/2) = 1 + T(n/4)$,
  and $T(n/4) = 1 + T(n/8)$, etc.
- So $T(n) = 1 + T(n/2) = 1 + 1 + T(n/4) = 1 + 1 + 1 + T(n/8)$
  and the pattern is $T(n) = k + T(n/2^k)$
  now when $k = \log n$, then $n/(2^{\log n}) = n/n = 1$,
- **$T(n) = \log n + 1 =$ $O(\log n)$**

Binary Search algorithm Analysis

- Best case time complexity $T(n)=O(1)$
- Worst case time complexity $T(n)=O(\log n)$
- Average case time complexity $T(n)=O(\log n)$

Finding the Maximum/Minimum

- Given an array of size n, you need to find the maximum and minimum element present in the array.
- In this approach, Divide the array into two equal parts and recursively find the maximum and minimum of those parts.
- After this, compare the maximum and minimum of those parts to get the maximum and minimum of the whole array.

```
Algorithm MaxMin(int A[], int n)
{
max = A[0] ;
 min = A[0] ;
for ( i = 1 to n-1 )
{
if ( A[i] > max )
max = A[i] ;
else if ( A[i] < min )
 min = A[i] ;
}
```

MaxMin requires 2(n-1) element comparisons in the best, average & worst cases.

```
Algorithm MaxMin(a,i,j,max,min)
 {
 if ( i==j ) { max = min=a[i];} //small p
else if ( i +1== j)//small p
{
        if ( a[i] < a[j] )
        {
        max = a[j] ;
         min = a[i] ;}
         else {
        max = a[i] ;
         min = a[j] ;}
         }
    }
```

```
else   //if p is not small,divide p into sub problems
 {
 int mid = i+j/2;
 MaxMin(a,i,mid,max,min);
 MaxMin(a,mid+1,j,max1,min1)
if ( max < max1 )
 max = max1;
 if ( min> min1 )
 min = min1;
}
}
```

Time complexity

$$T(n)=\begin{cases}2T(n/2)+2 & n>2\\ 1 & n=2\end{cases}$$

$T(n) = 2T(n/2)+2$

$= 2(2T(n/4)+2)+2$

$= 4T(n/4)+4+2$

$\vdots$

$=2^k\,T(n/2k-1)+2^{k-1}+\ldots+4+2$

$=2^{k-1}+2^{k-1}+\ldots+4+2+1$

$=2^{k-1}+2^k-2=n/2+n-2=3n/2-2$

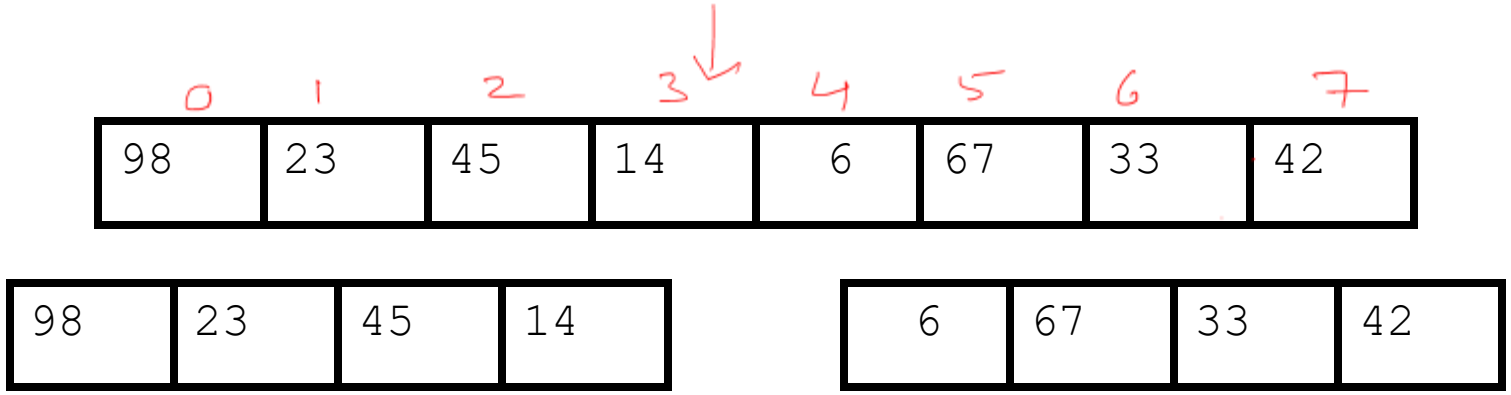
$T(n)=O(n)$

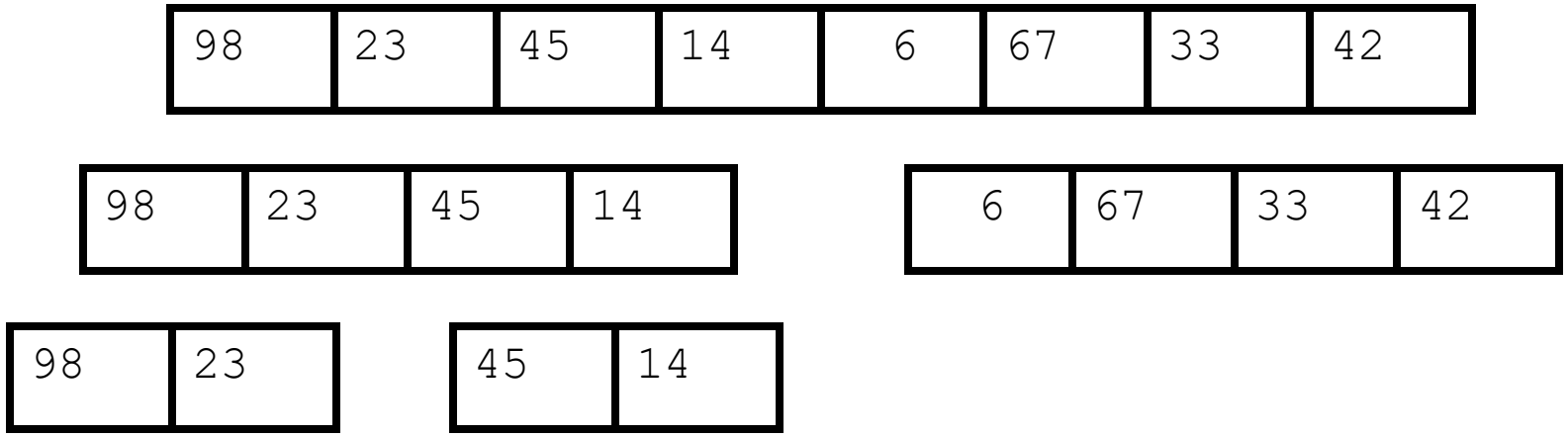
Merge sort

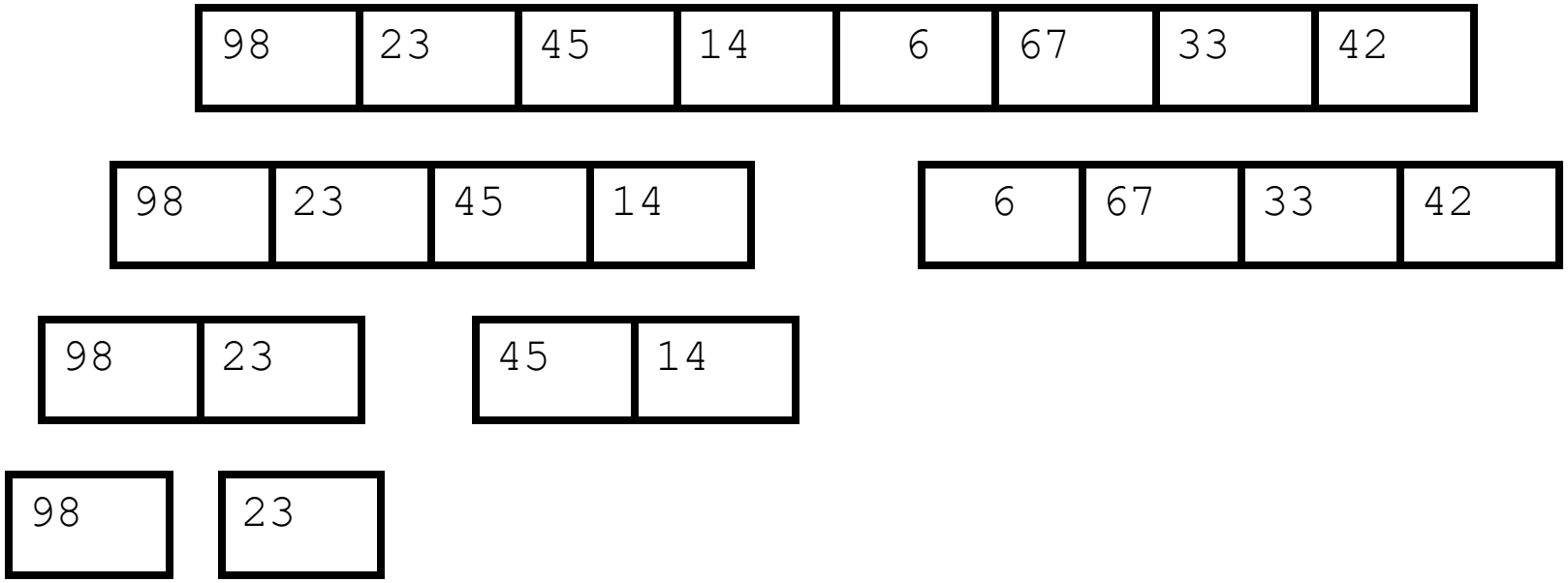
- Merge sort is a sorting algorithm that uses the divide and conquer strategy
- Divide the unsorted array into two sub lists at the middle.
- Further divide these sub lists into sub lists at their middles until each sub list contain only one element.
- Merge these sub lists two at a time till we get the final sorted list using the following steps:
  1. Compare the sub-list's first elements.
  2. Remove the smallest element and put it into the result array.
  3. Continue the process until all elements have been put into the result array.

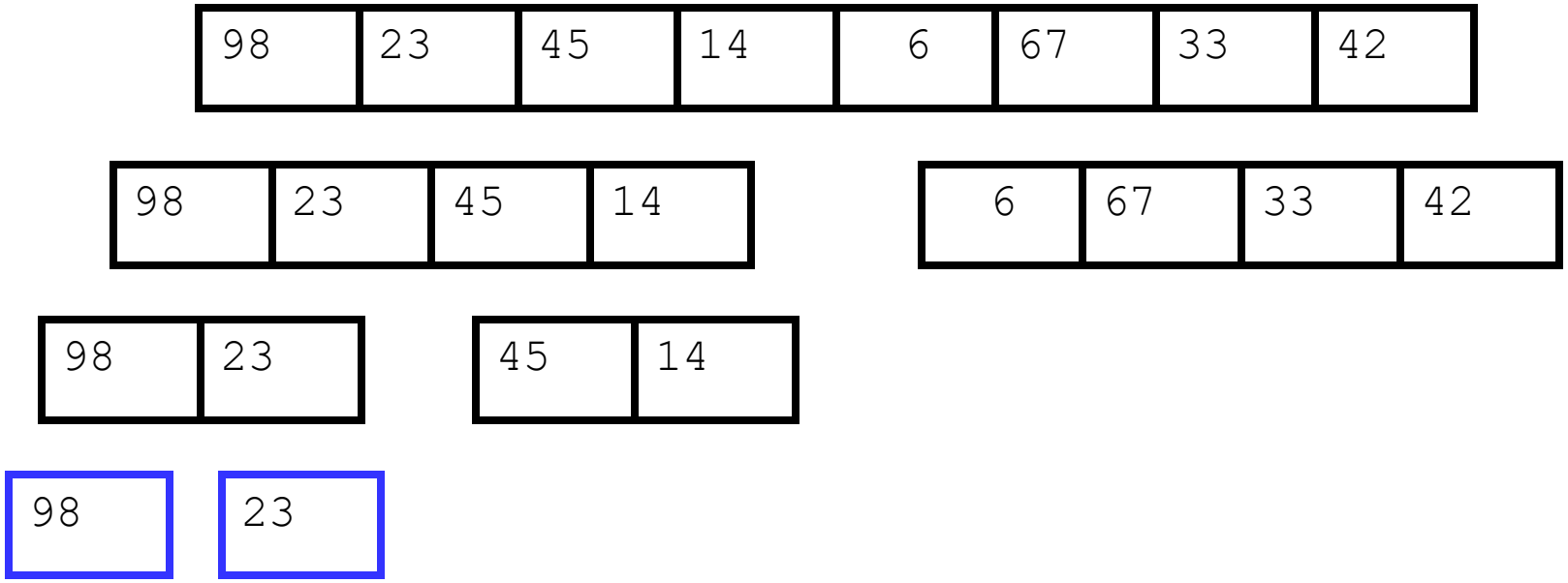
Merge sort

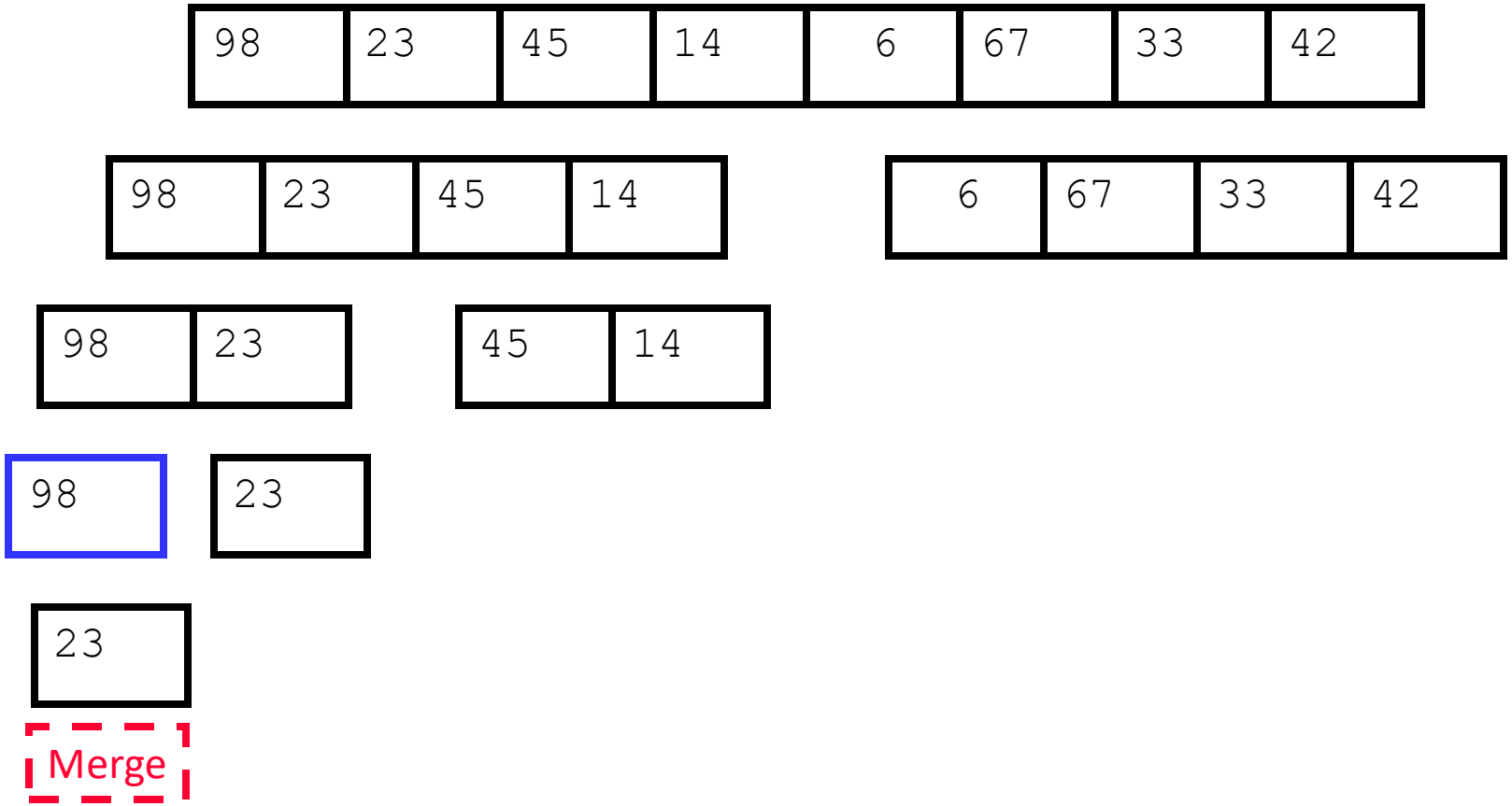
- Merge sort consists of three steps:
- 1.Partition array into two sub lists s1 and s2 with n/2 elements each.
- 2.Recursively sort s1 and s2.
- 3.Merge s1 and s2 into a single sorted list.

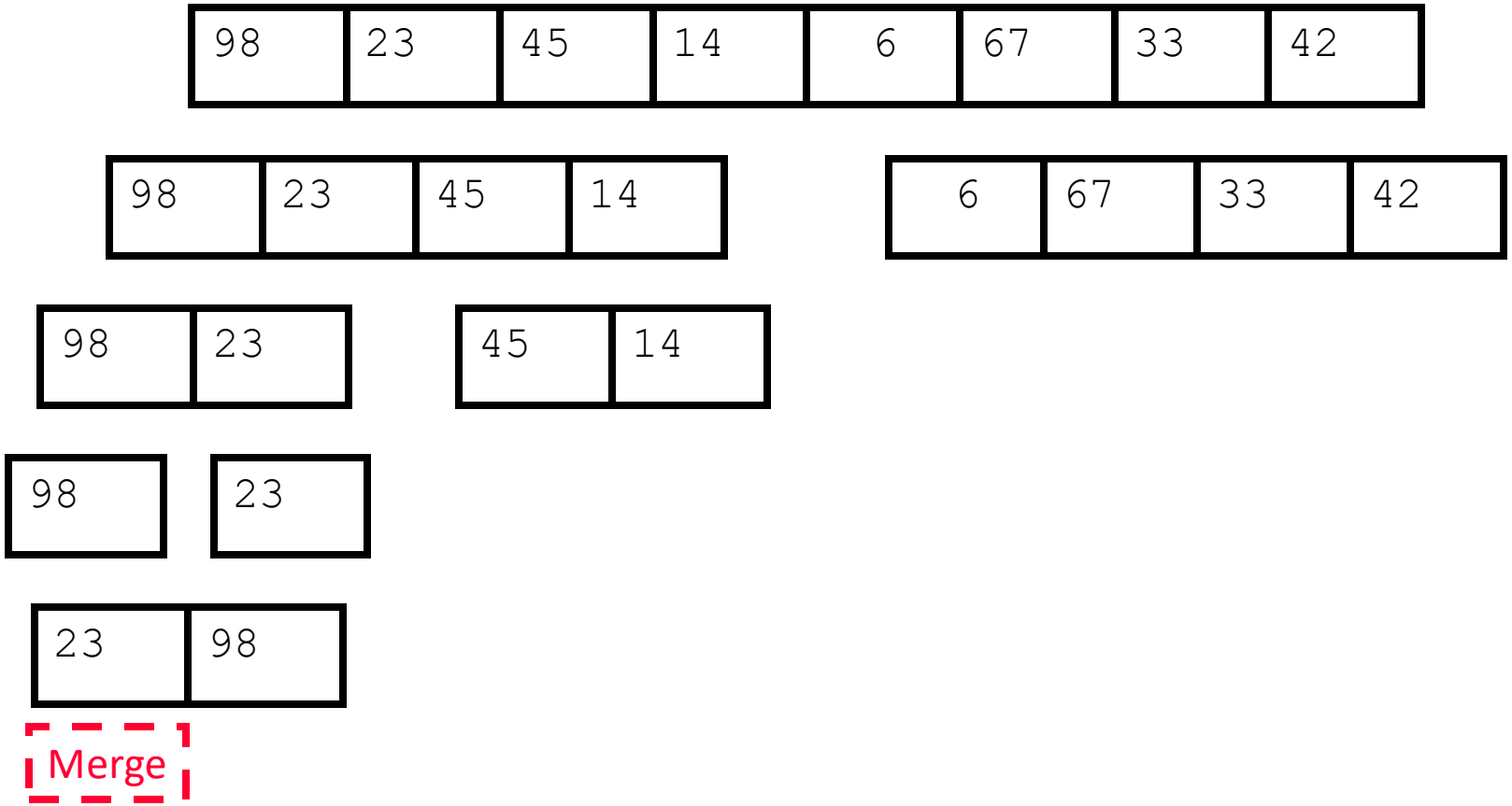

0
1
2
3
4
5
6
7
98
23
45
14
6
67
33
42
98
23
45
14
6
67
33
42

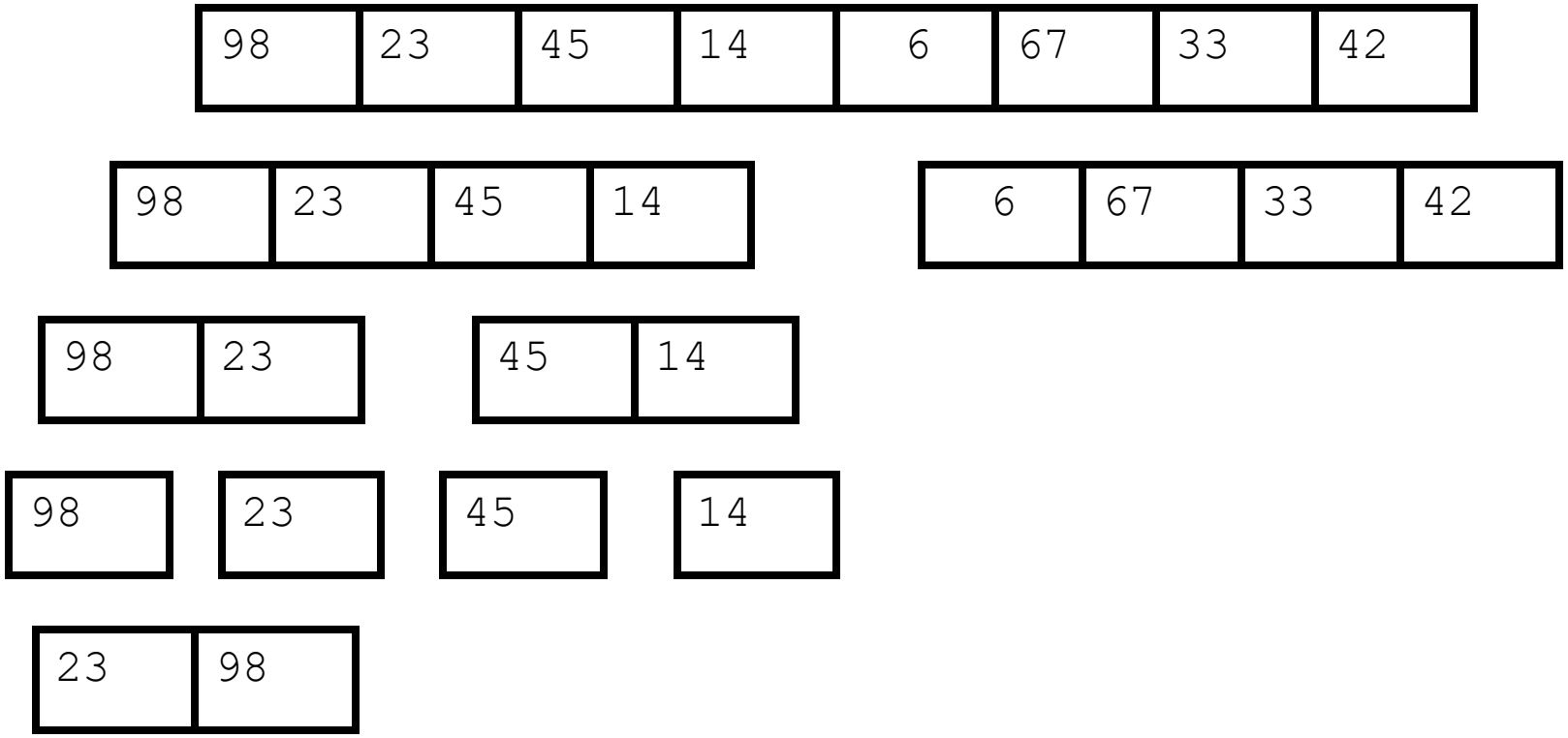

98
23
45
14
6
67
33
42
98
23
45
14
6
67
33
42
98
23
45
14

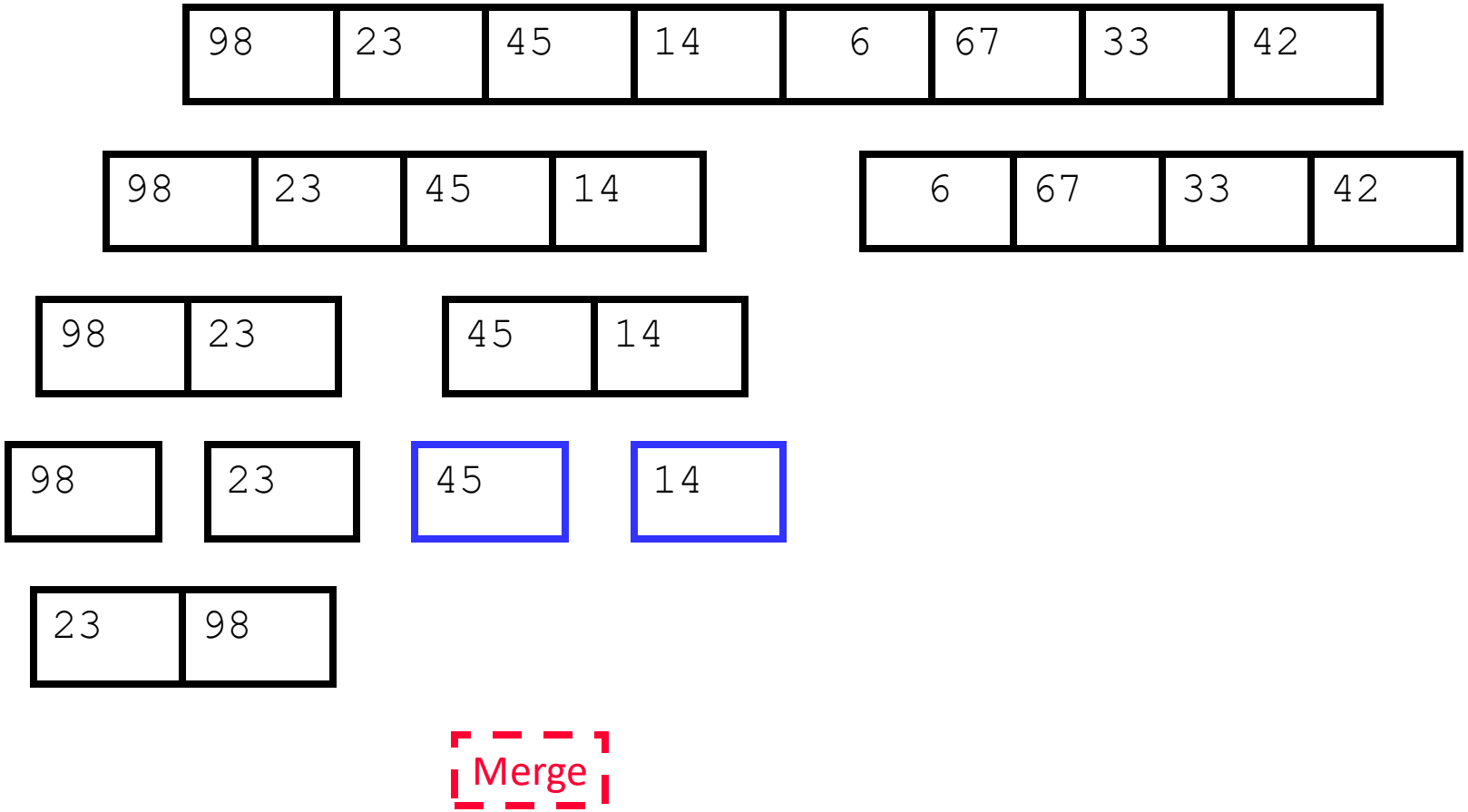

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23

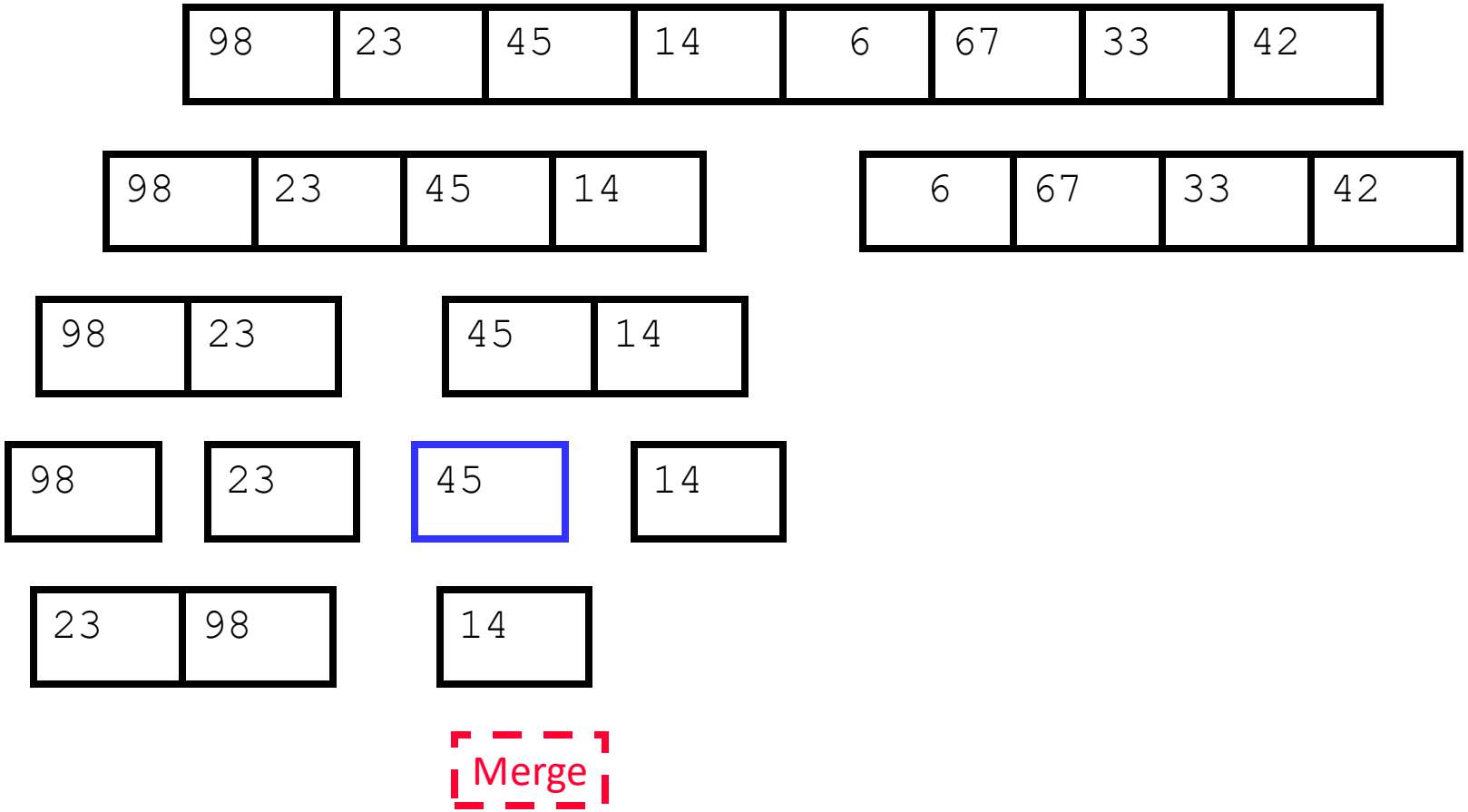

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
Merge

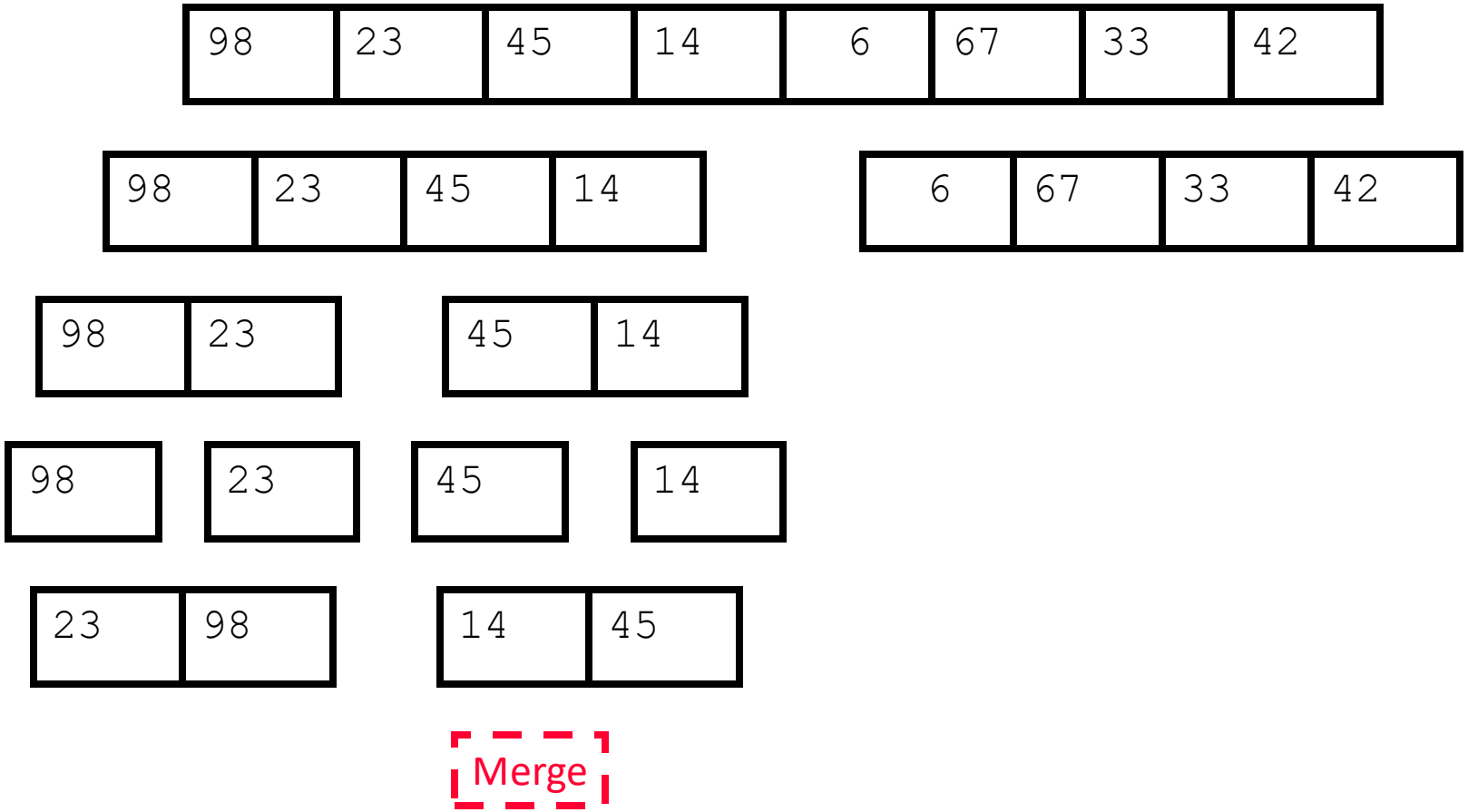

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23
Merge

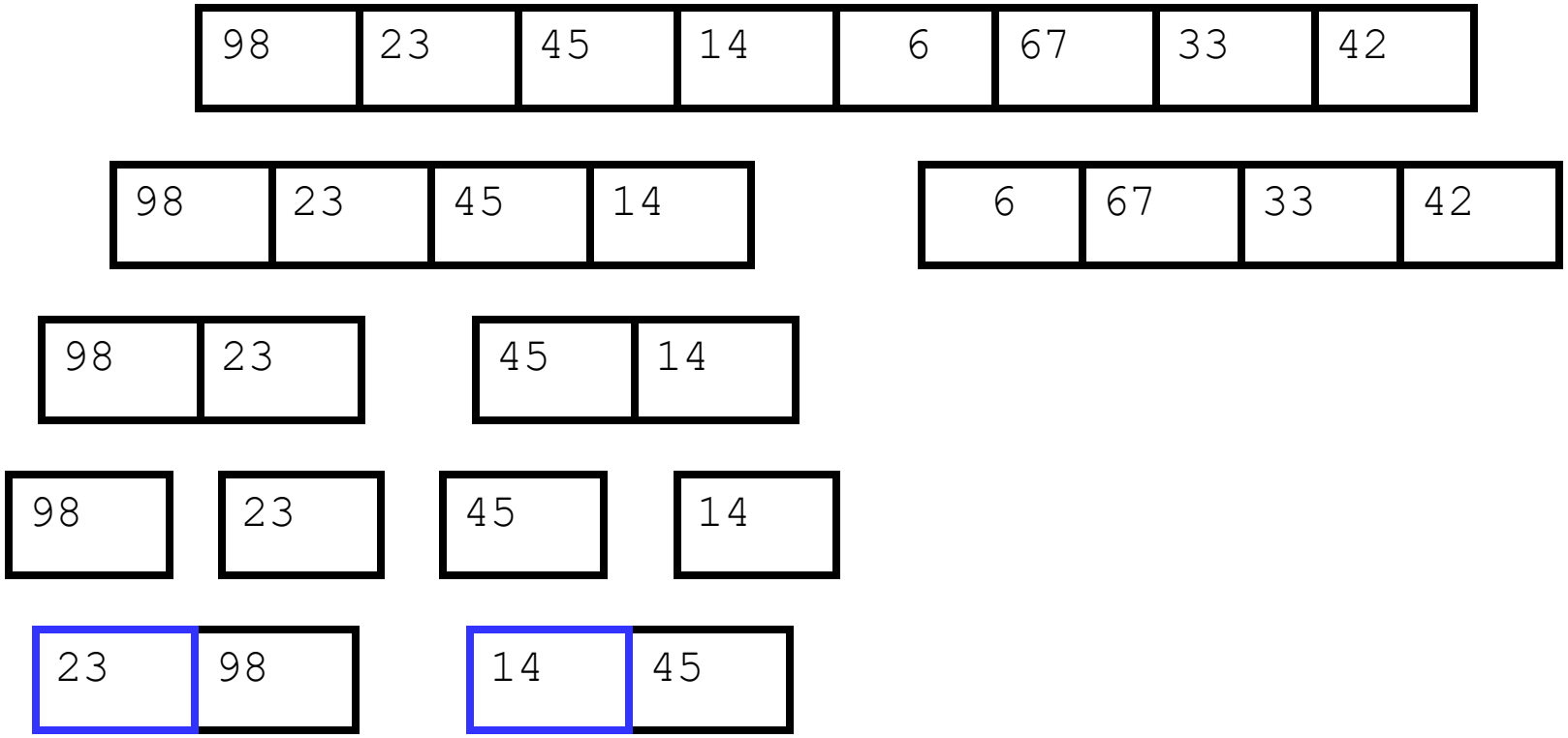

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23 98
Merge

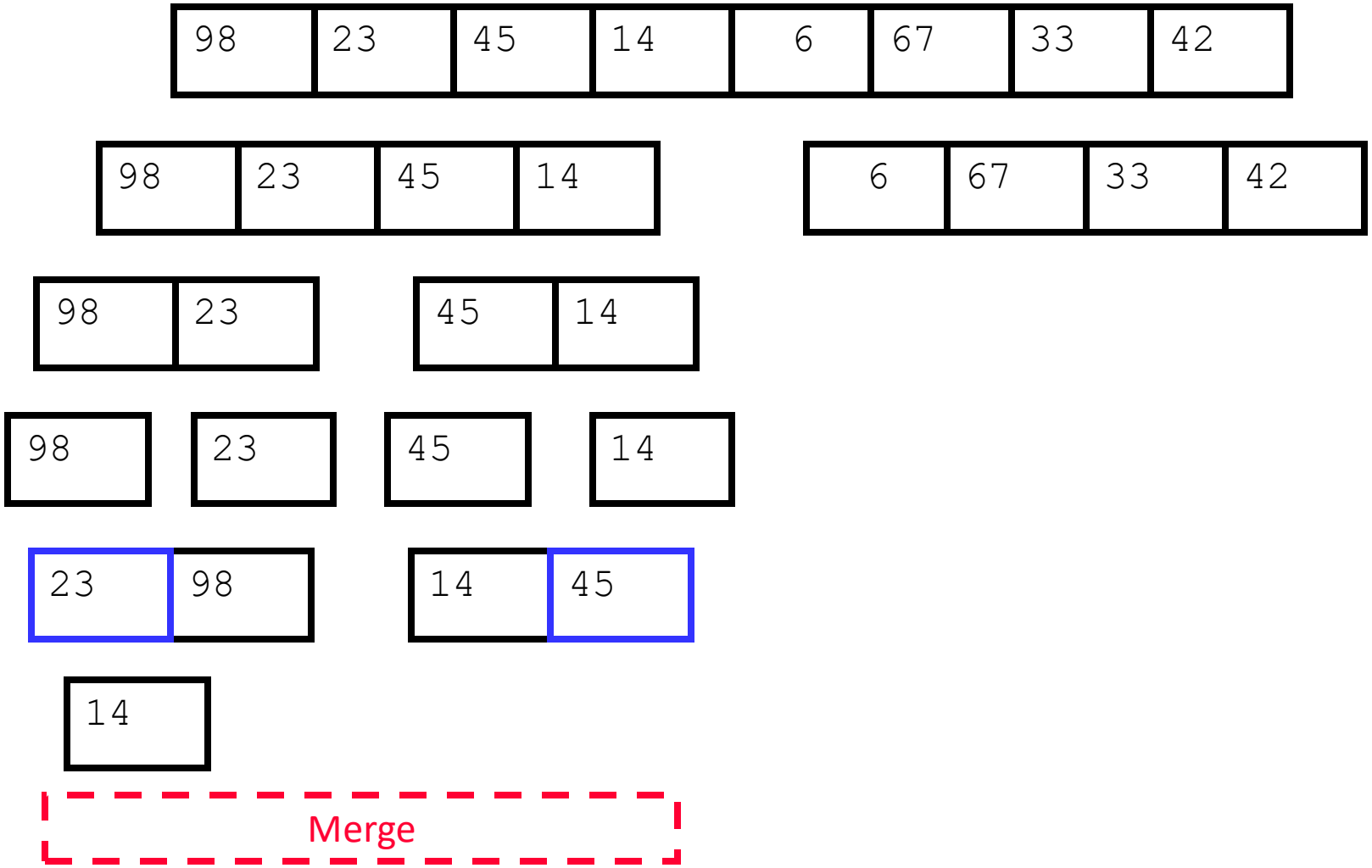

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98

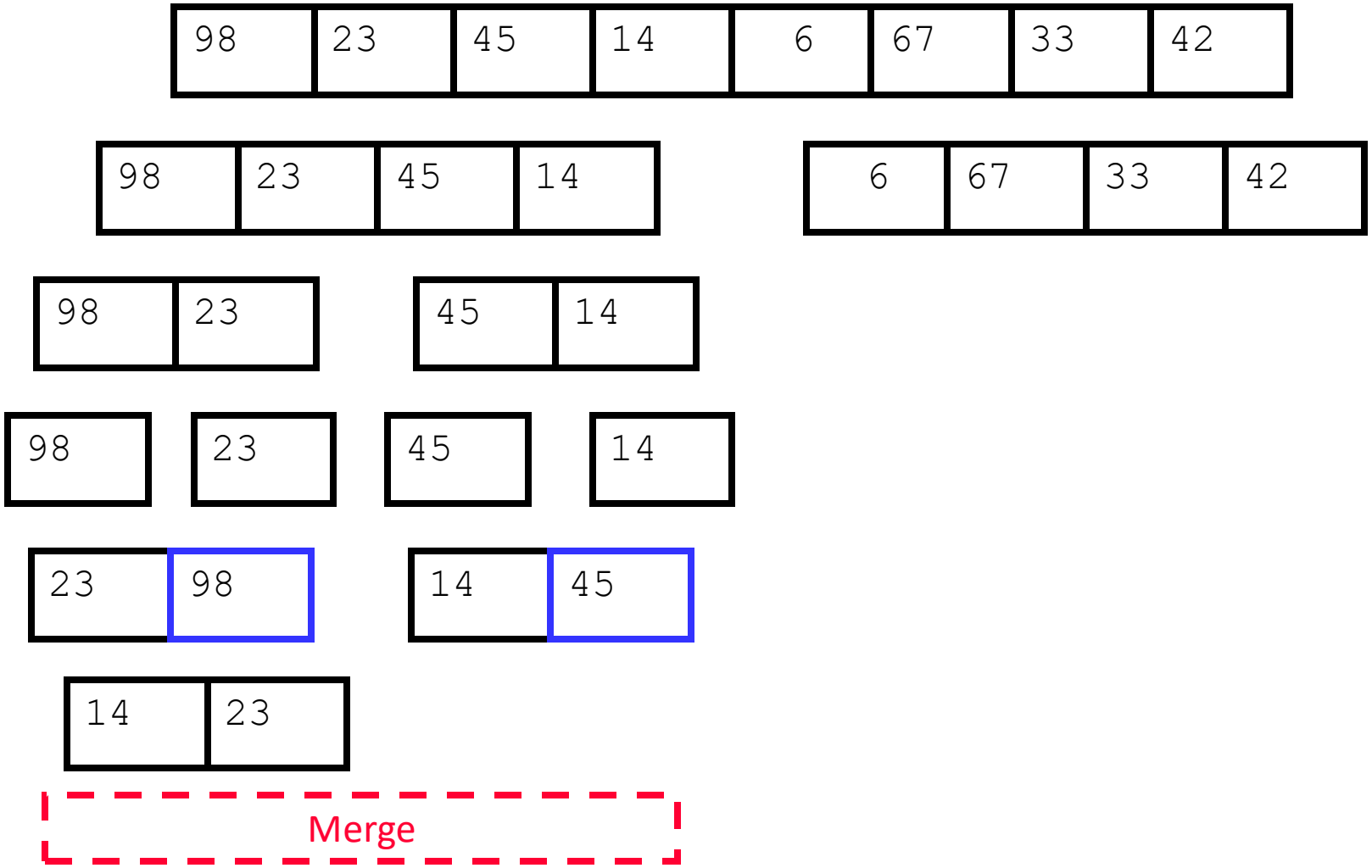

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
Merge

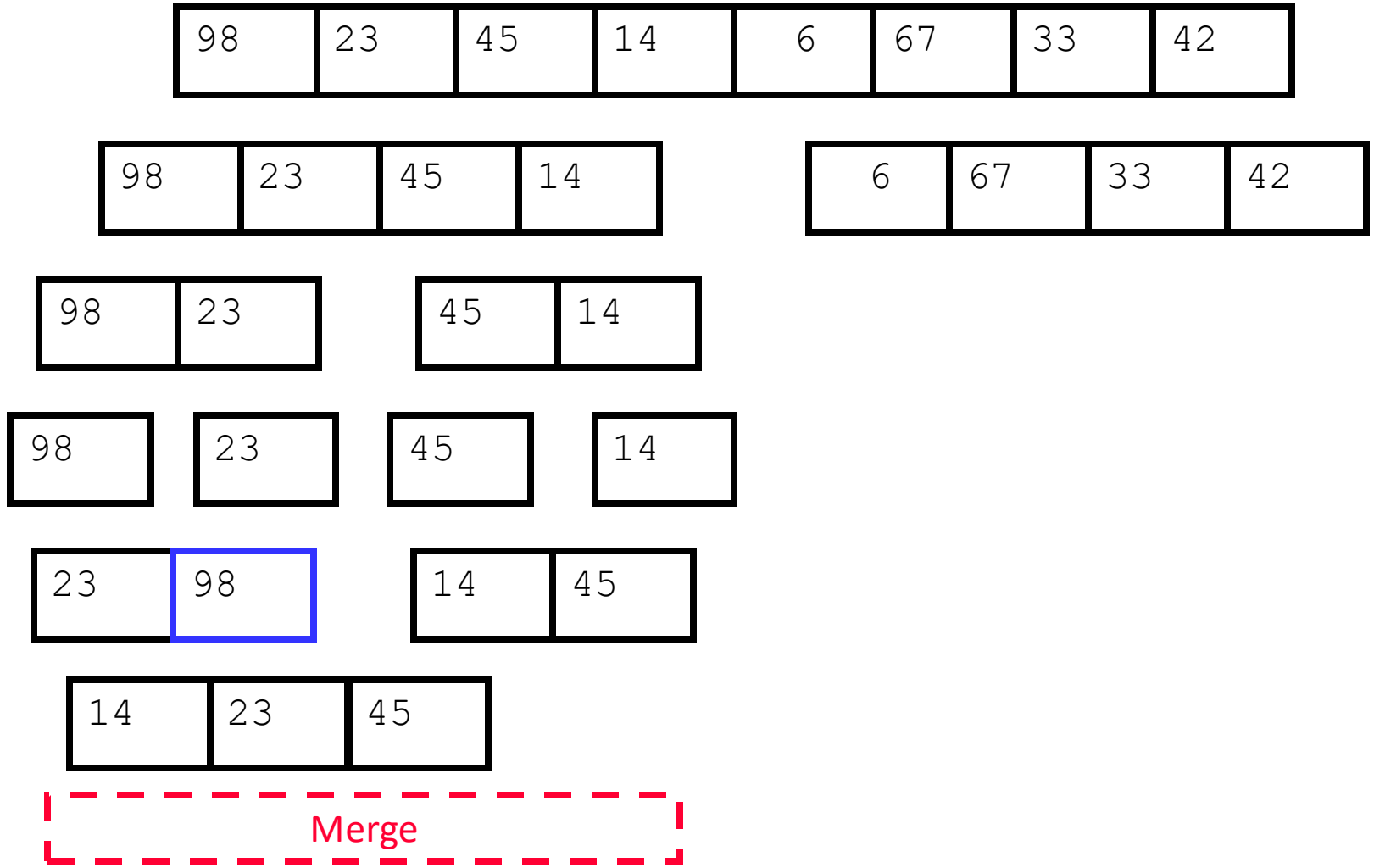

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14
Merge

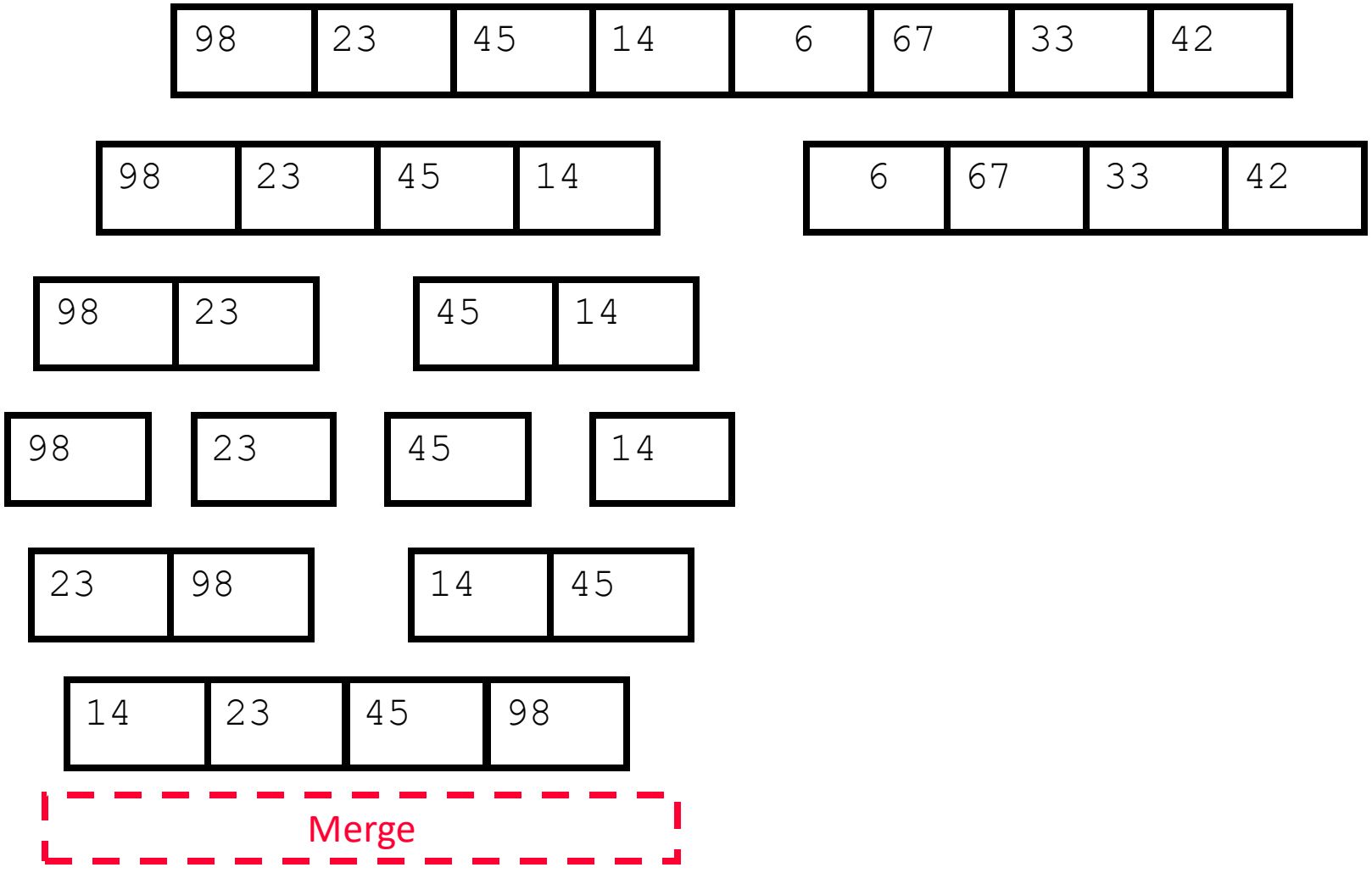

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

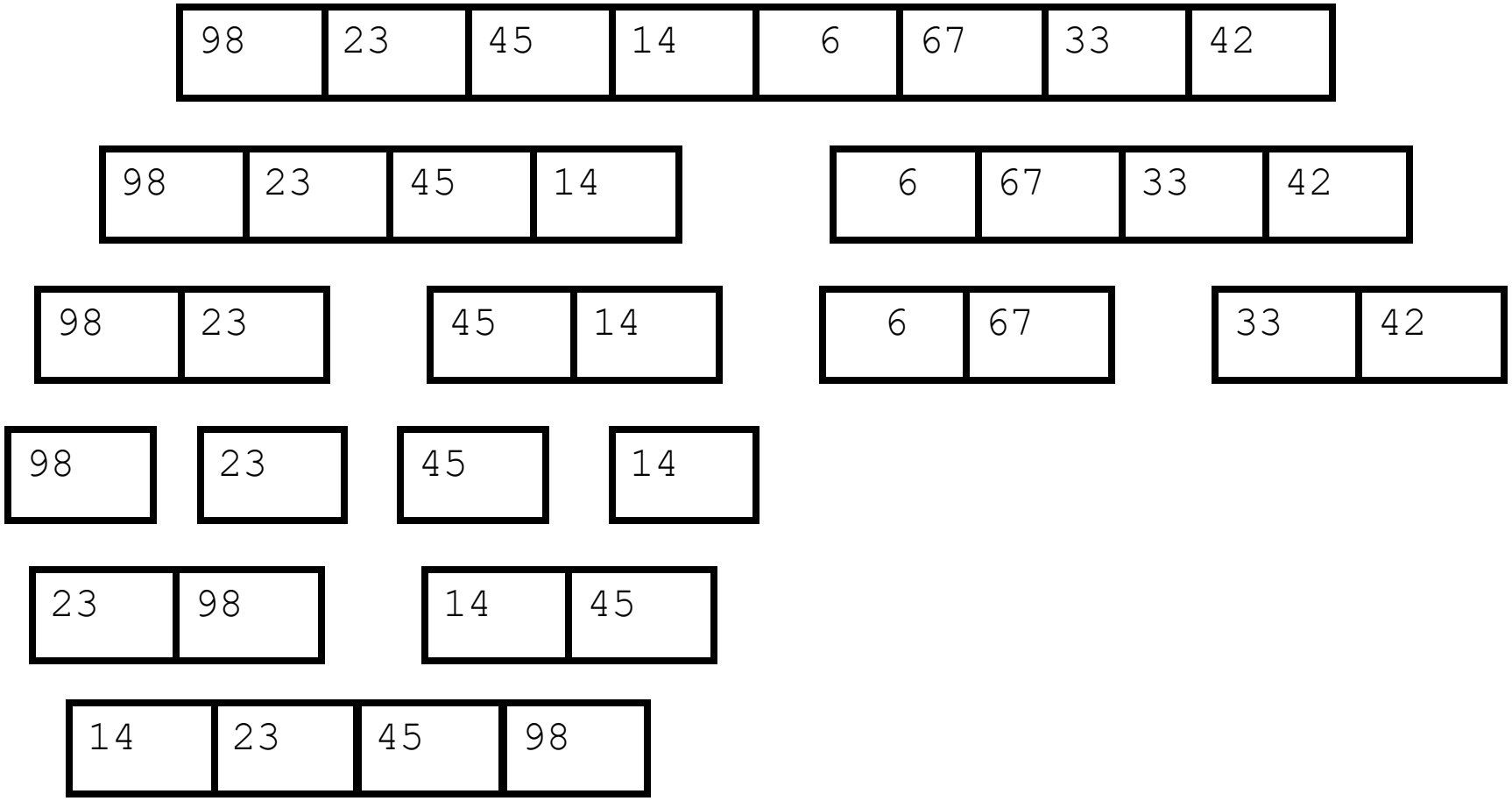

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

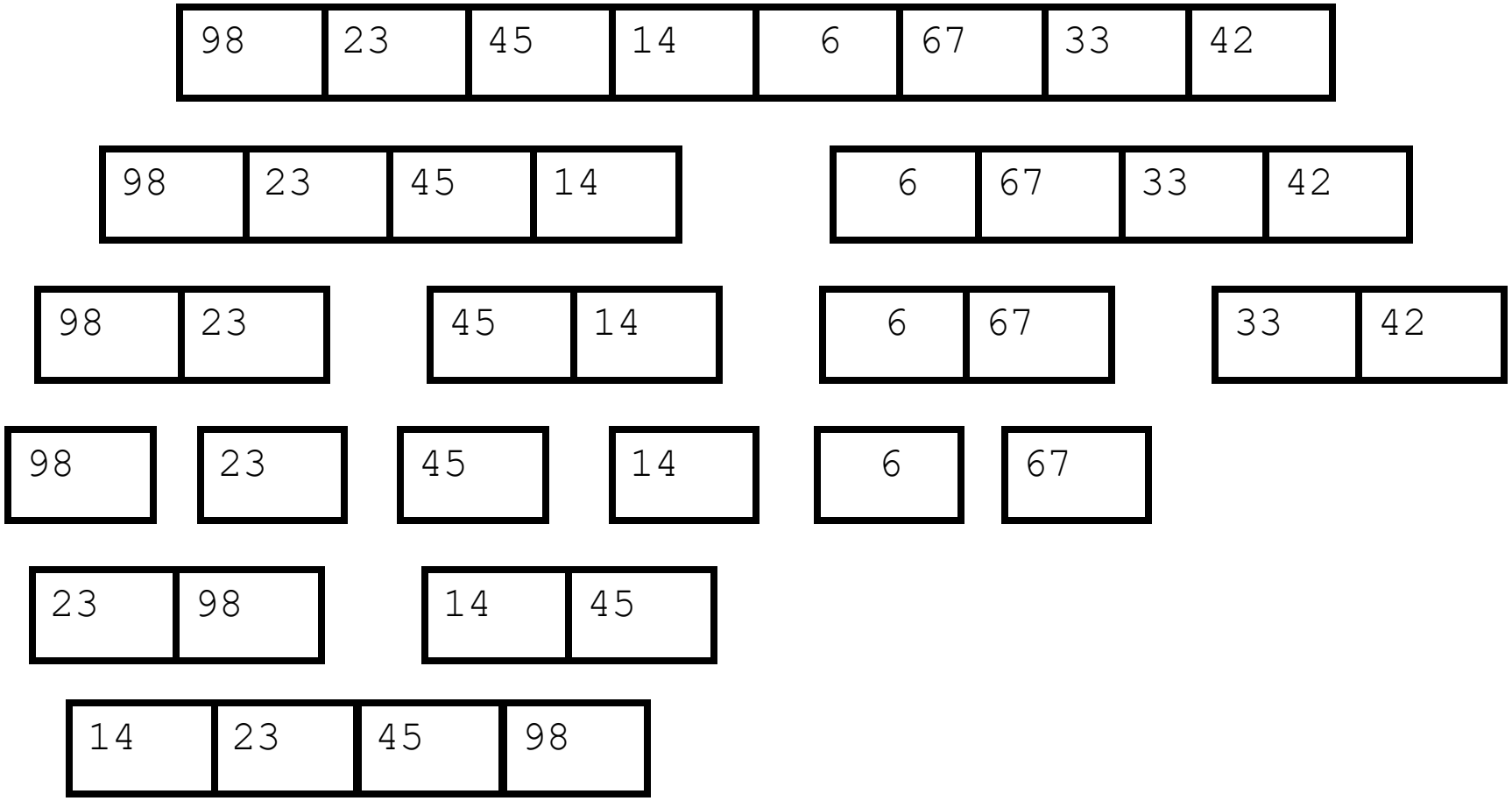

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14
Merge

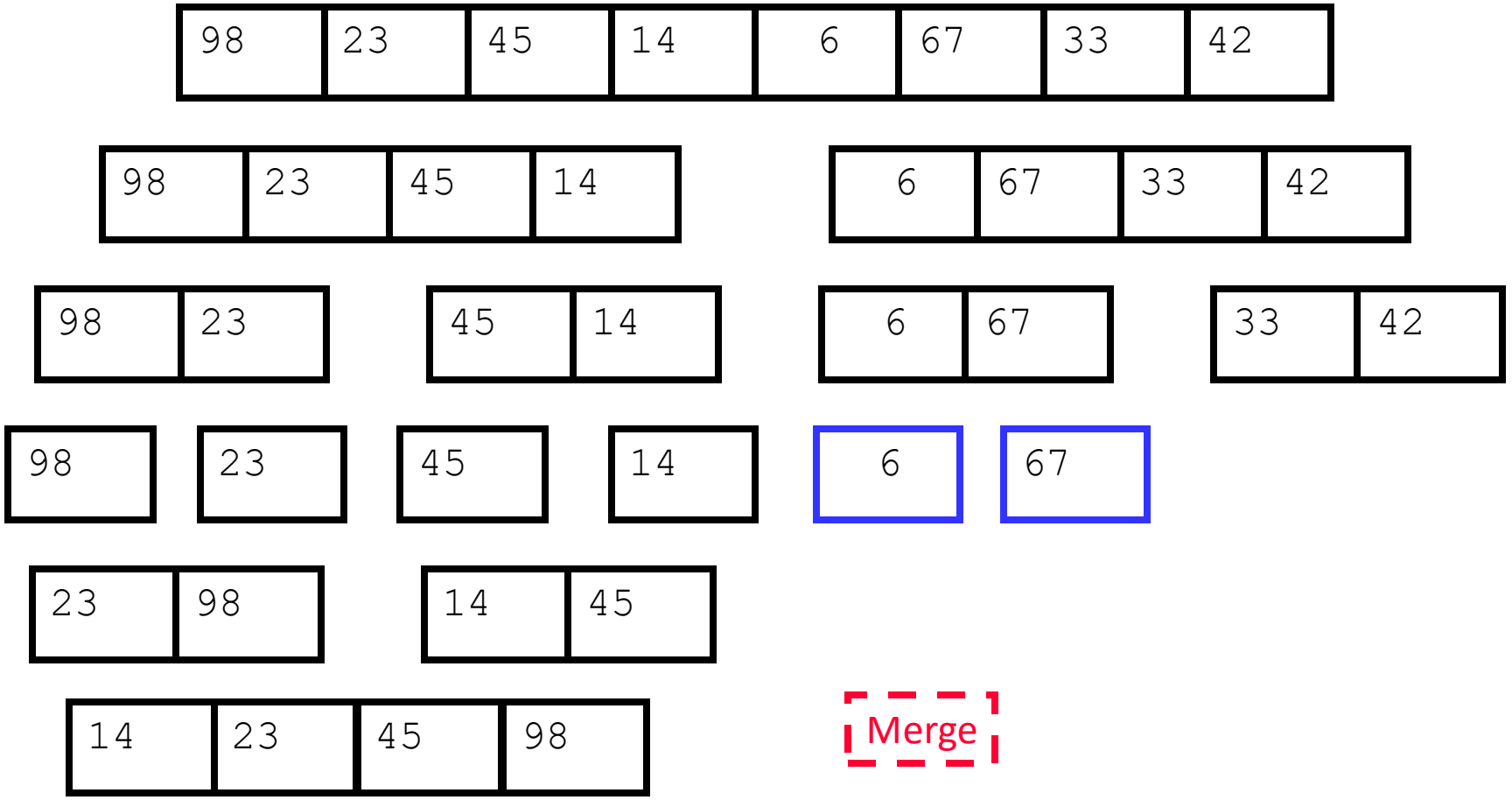

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23
Merge

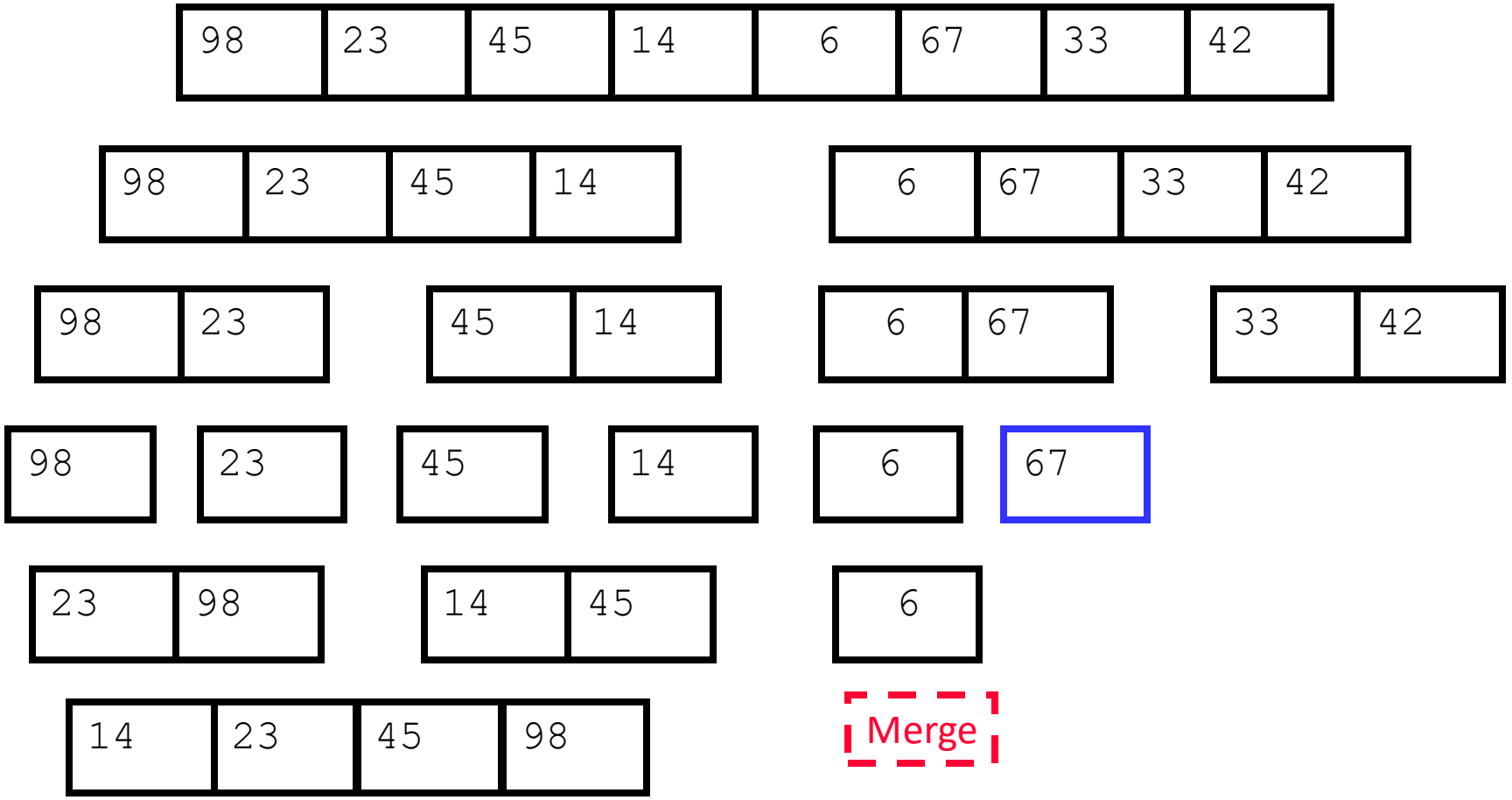

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45
Merge

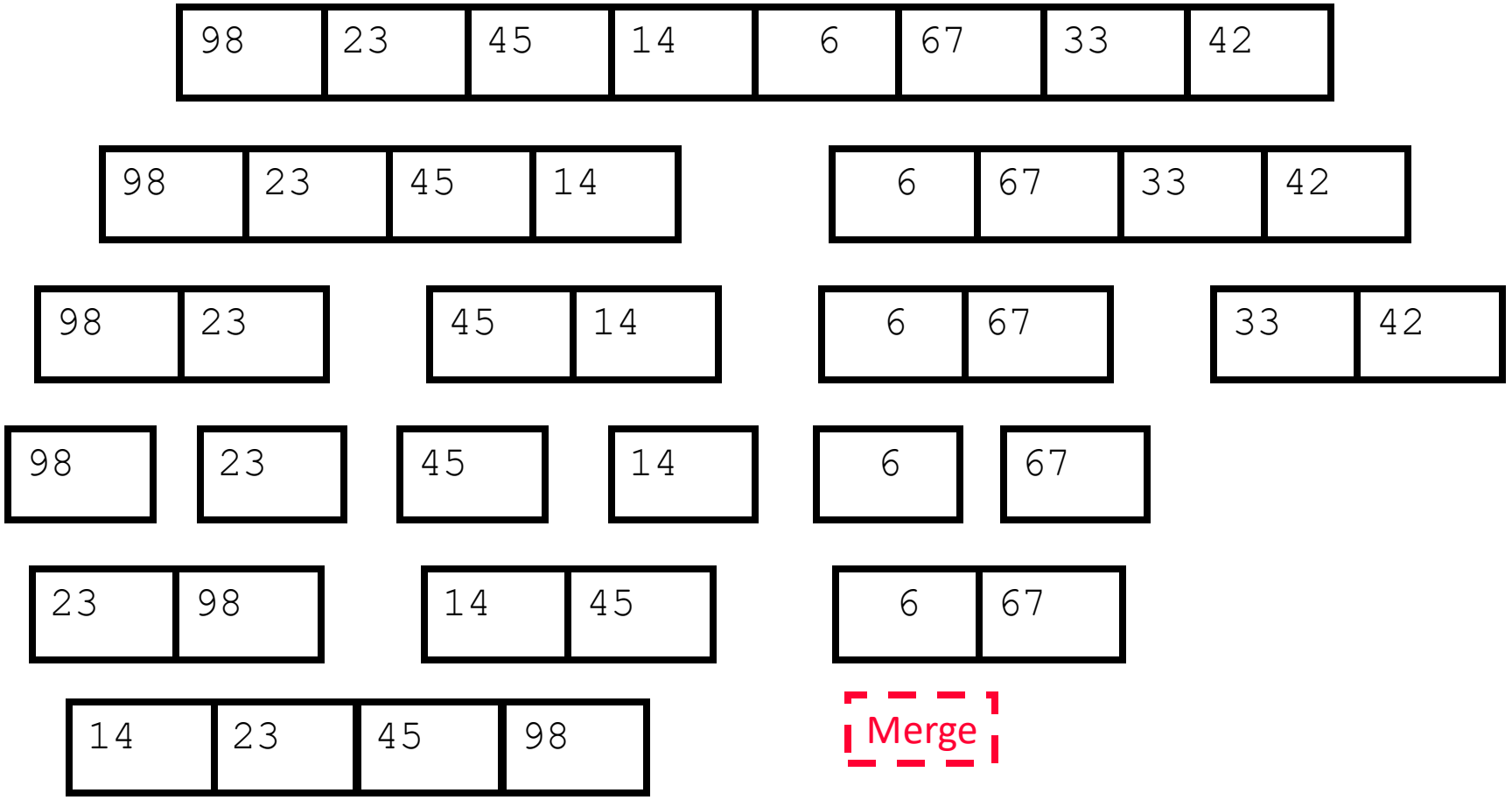

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45 98
Merge

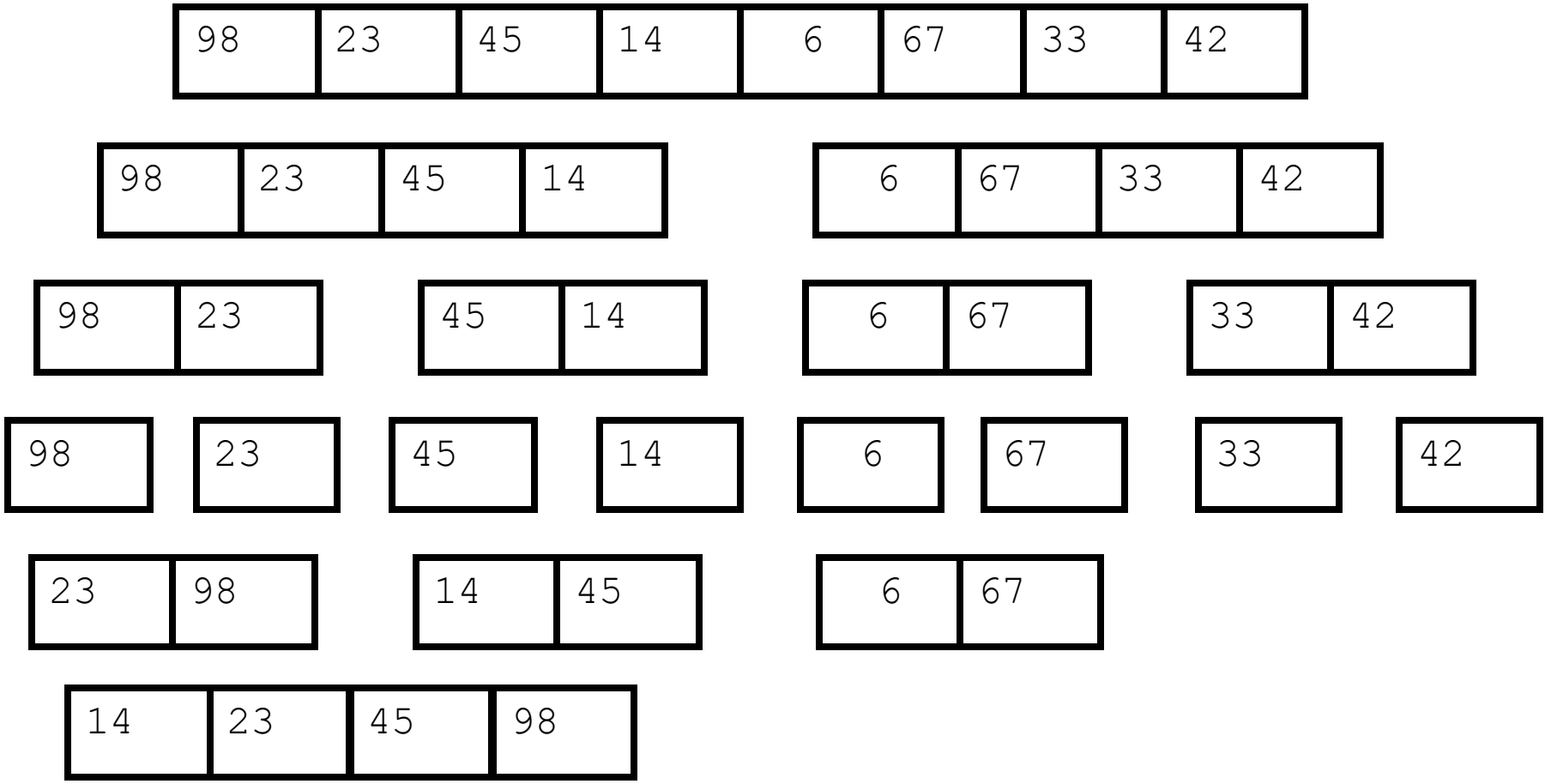

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
23 98
14 45
14 23 45 98

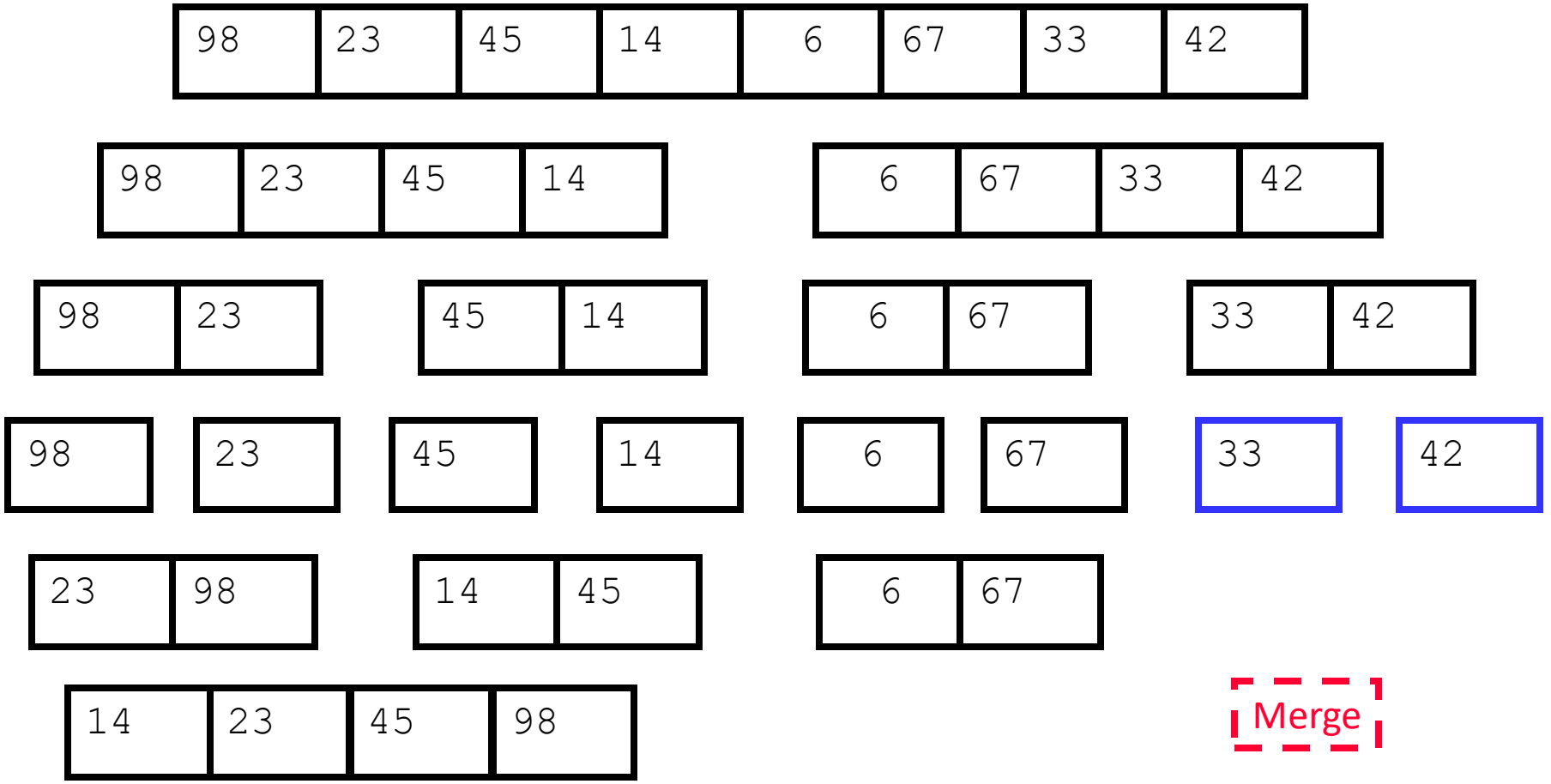

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98

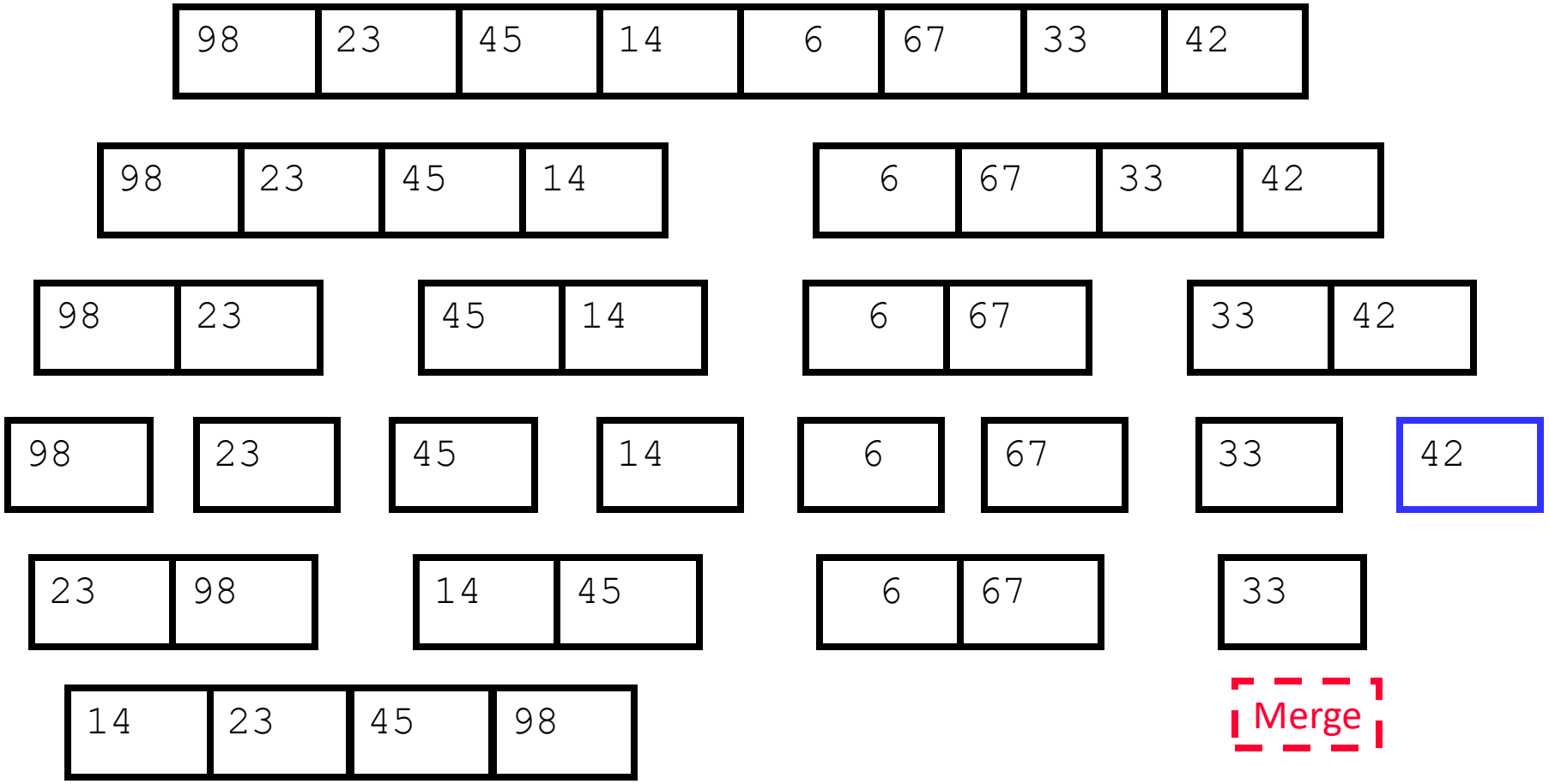

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98
Merge

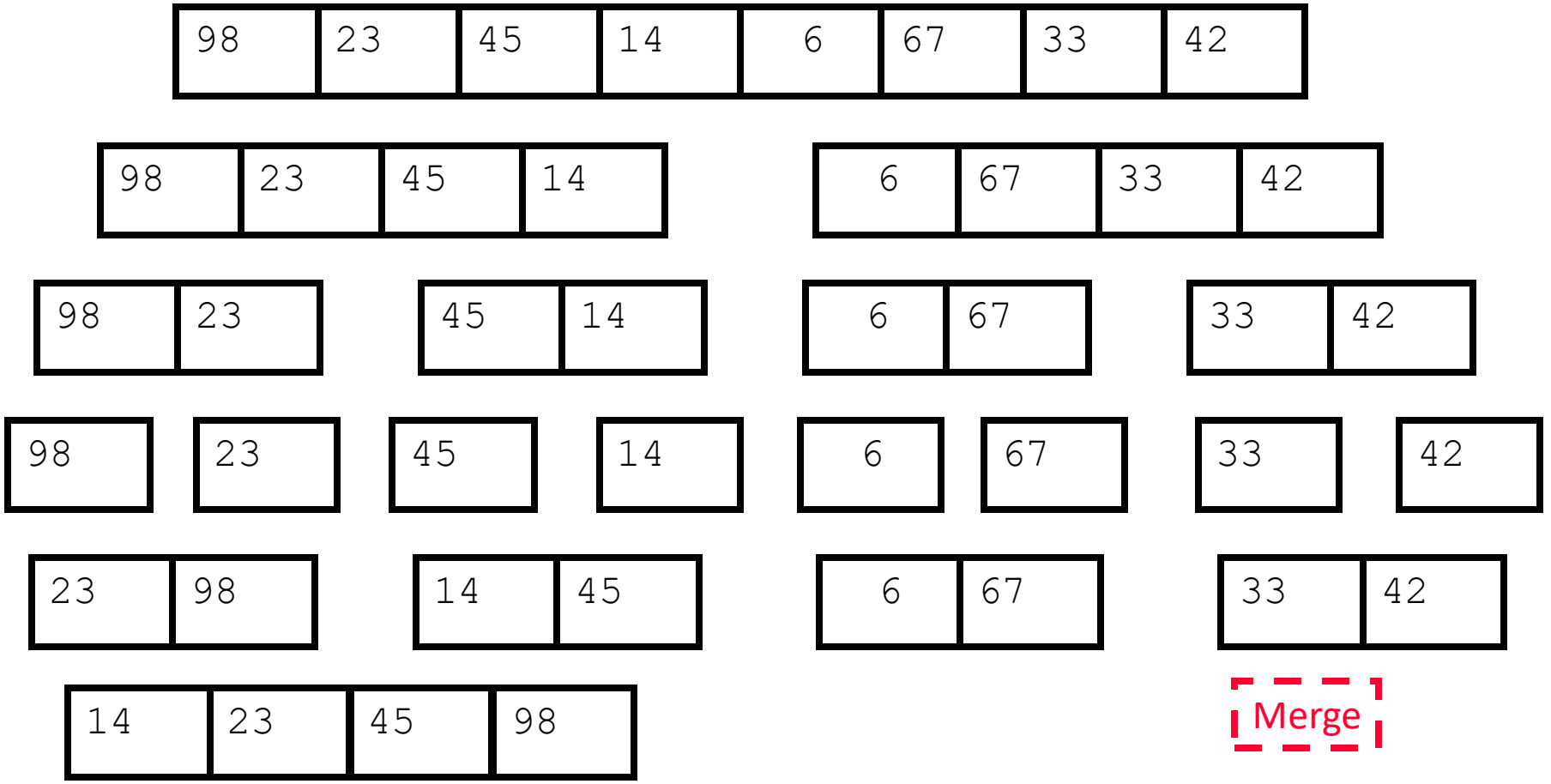

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6
14 23 45 98
Merge

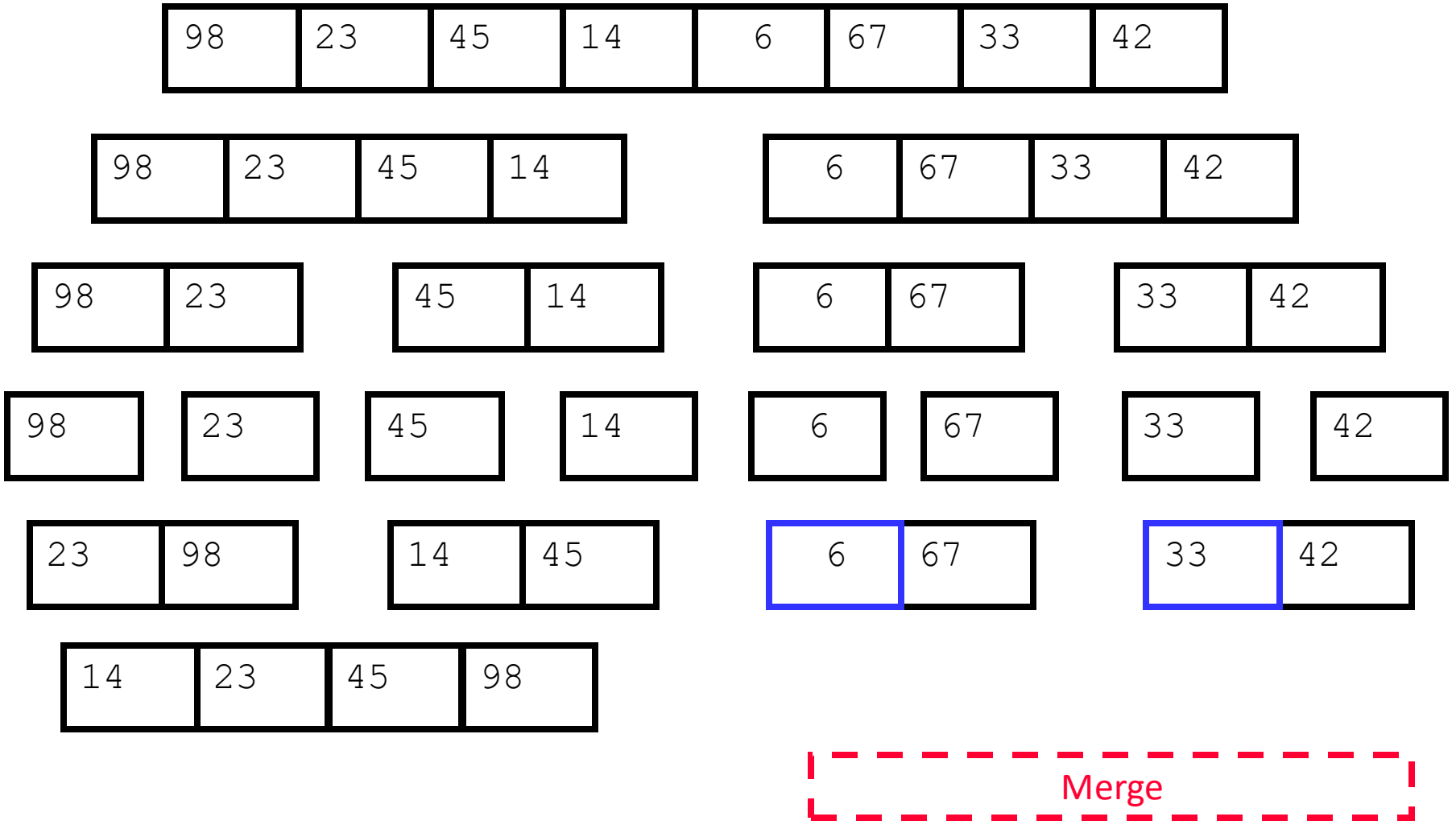

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6 67
14 23 45 98
Merge

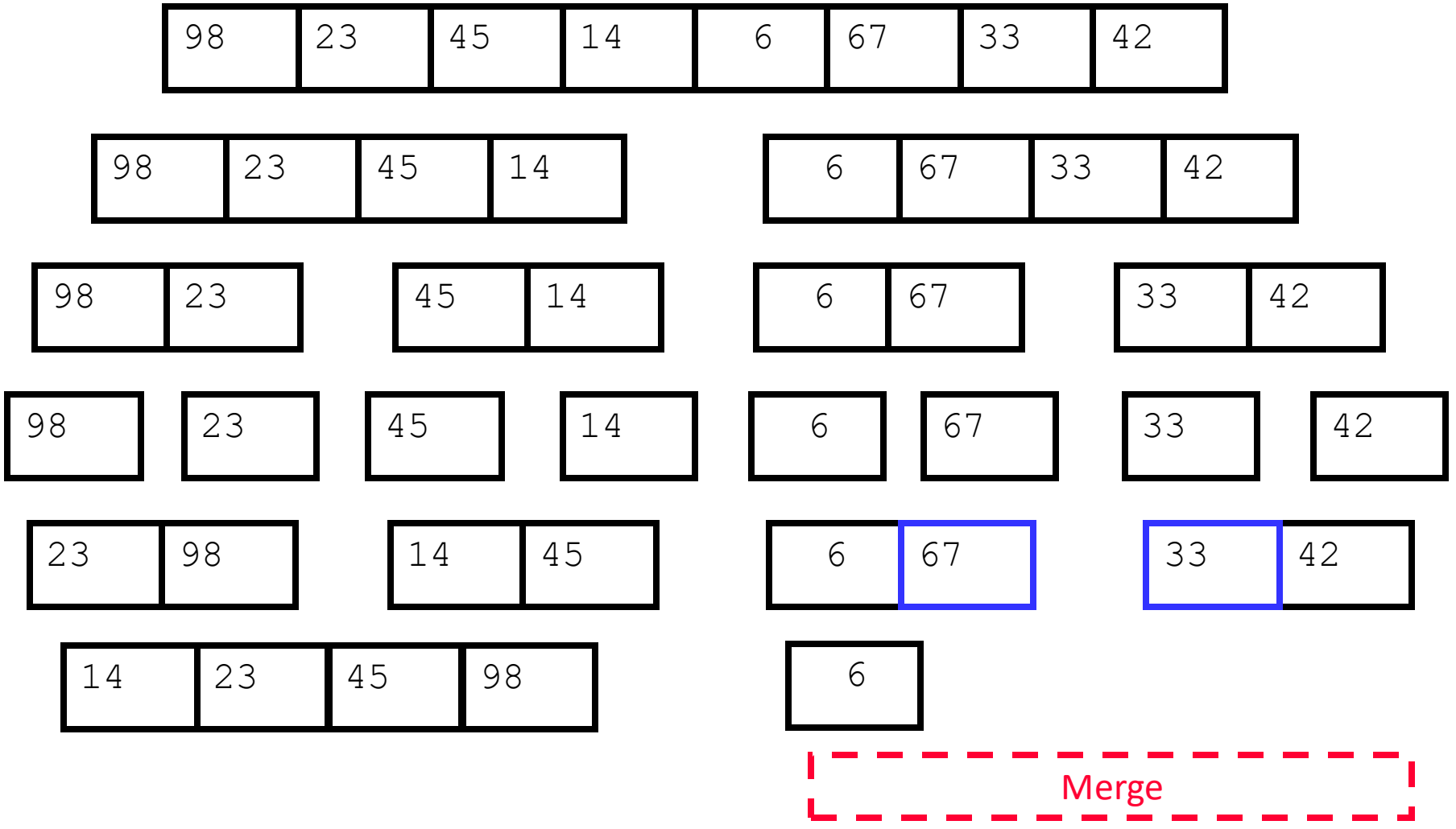

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98

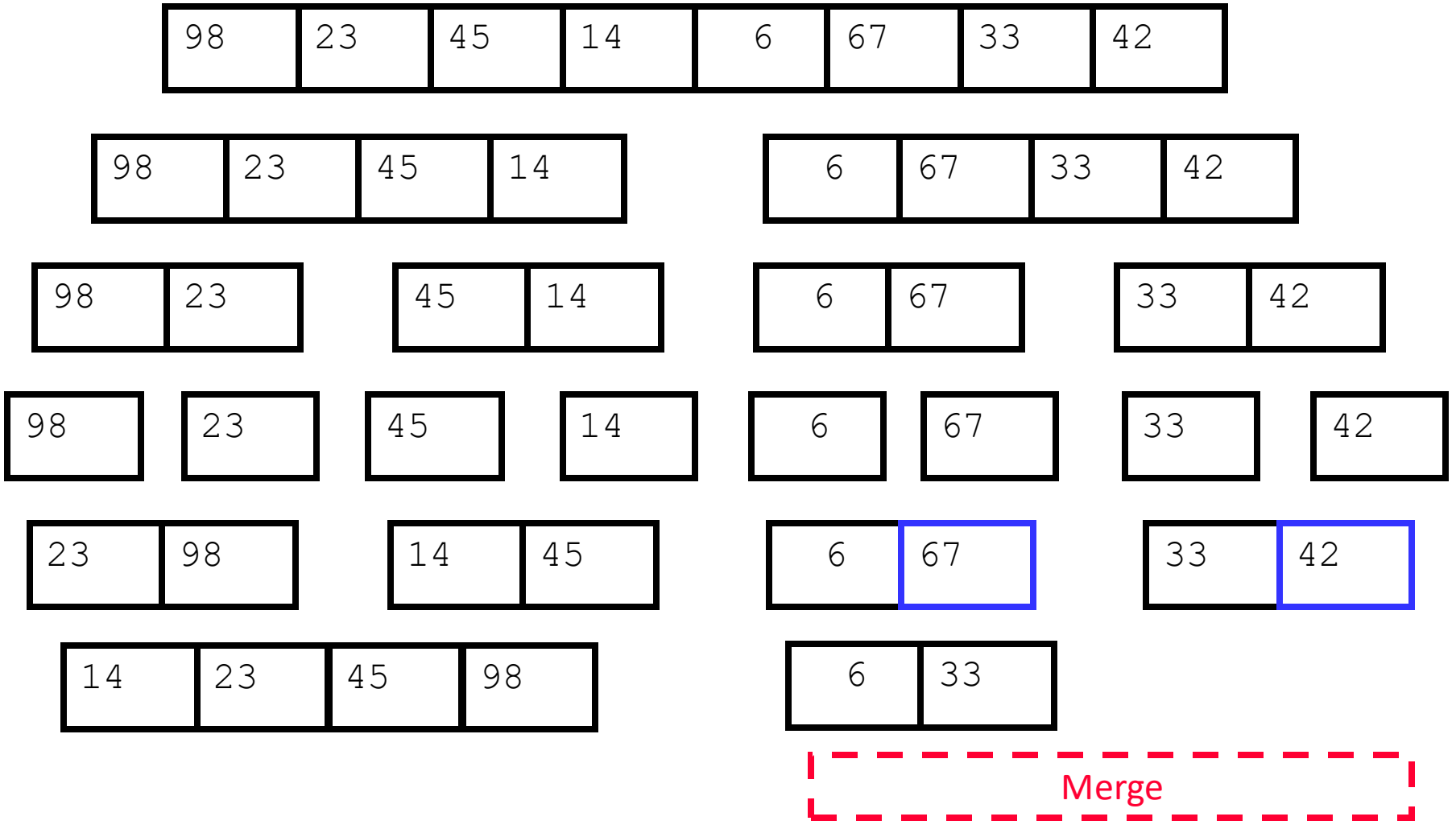

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98
Merge

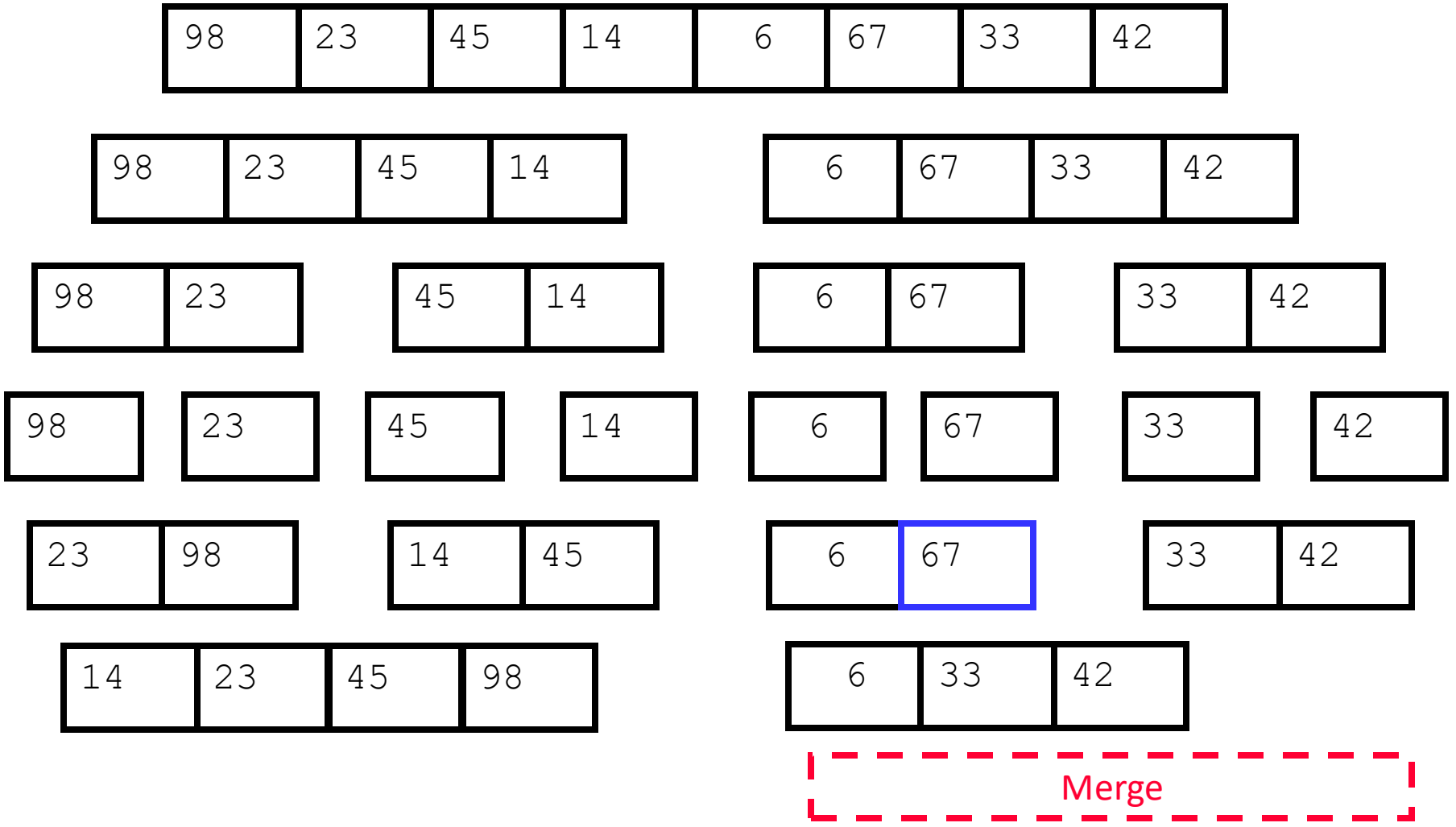

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33
14 23 45 98
Merge

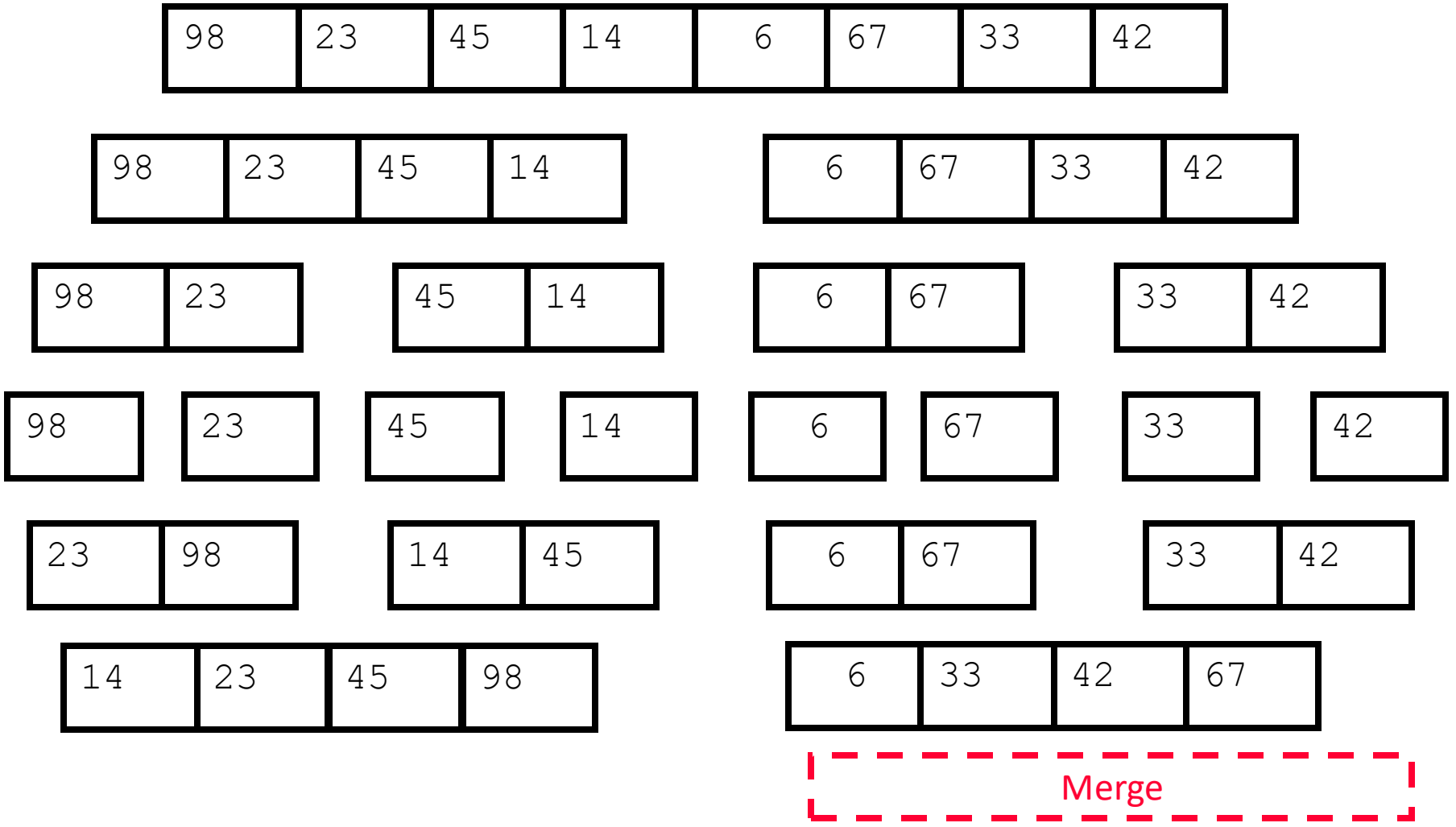

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge

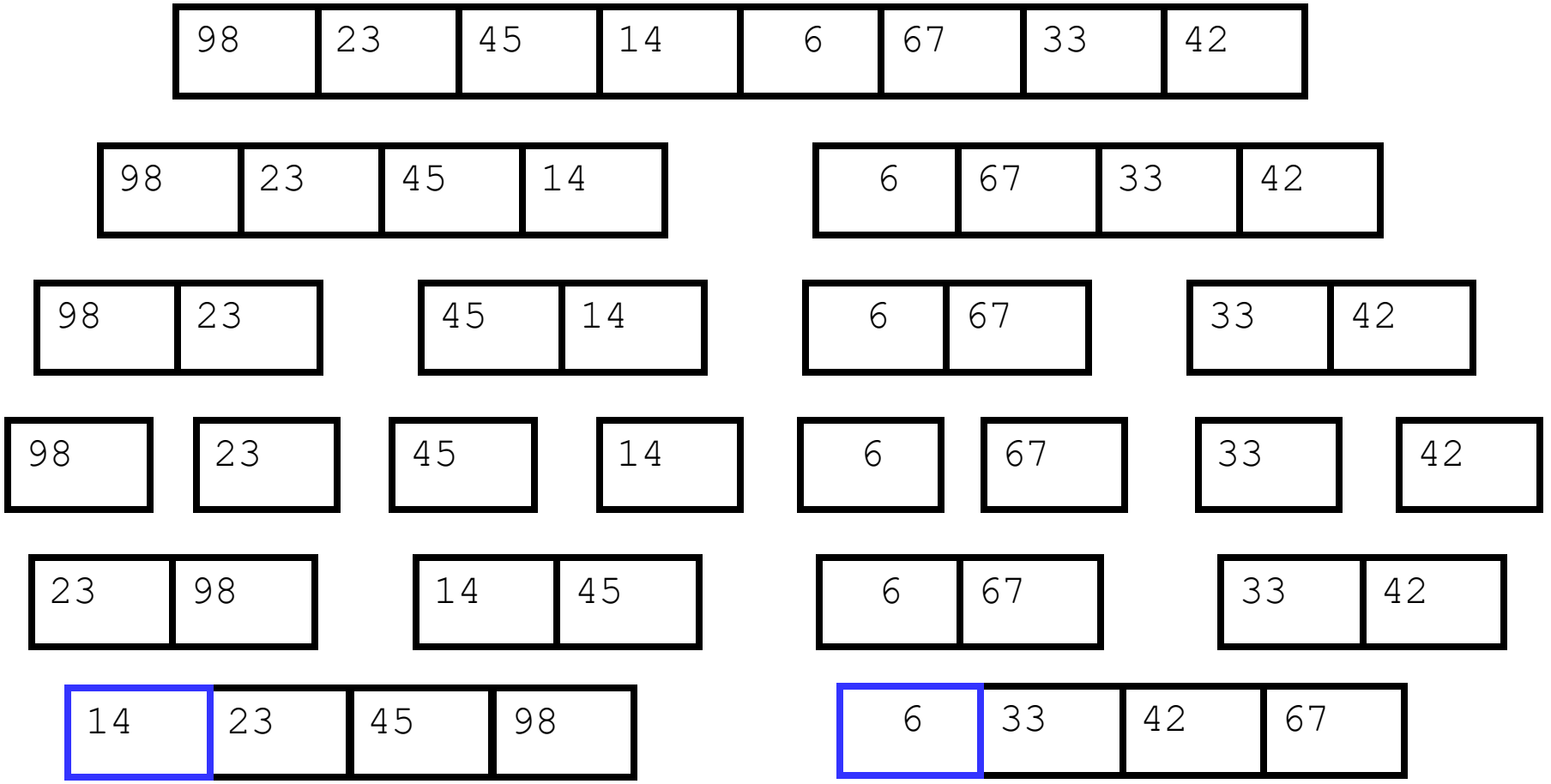

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge

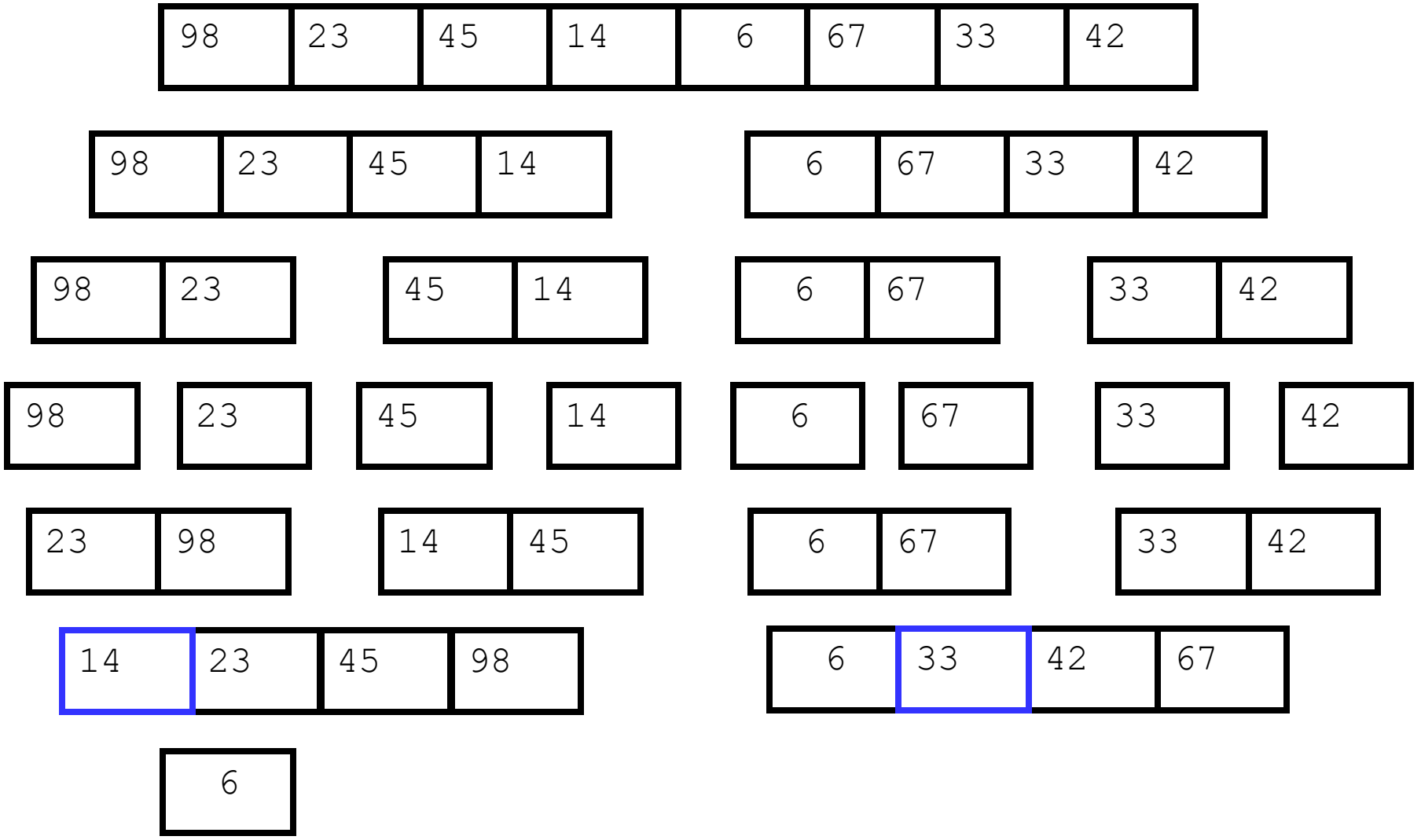

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6
Merge

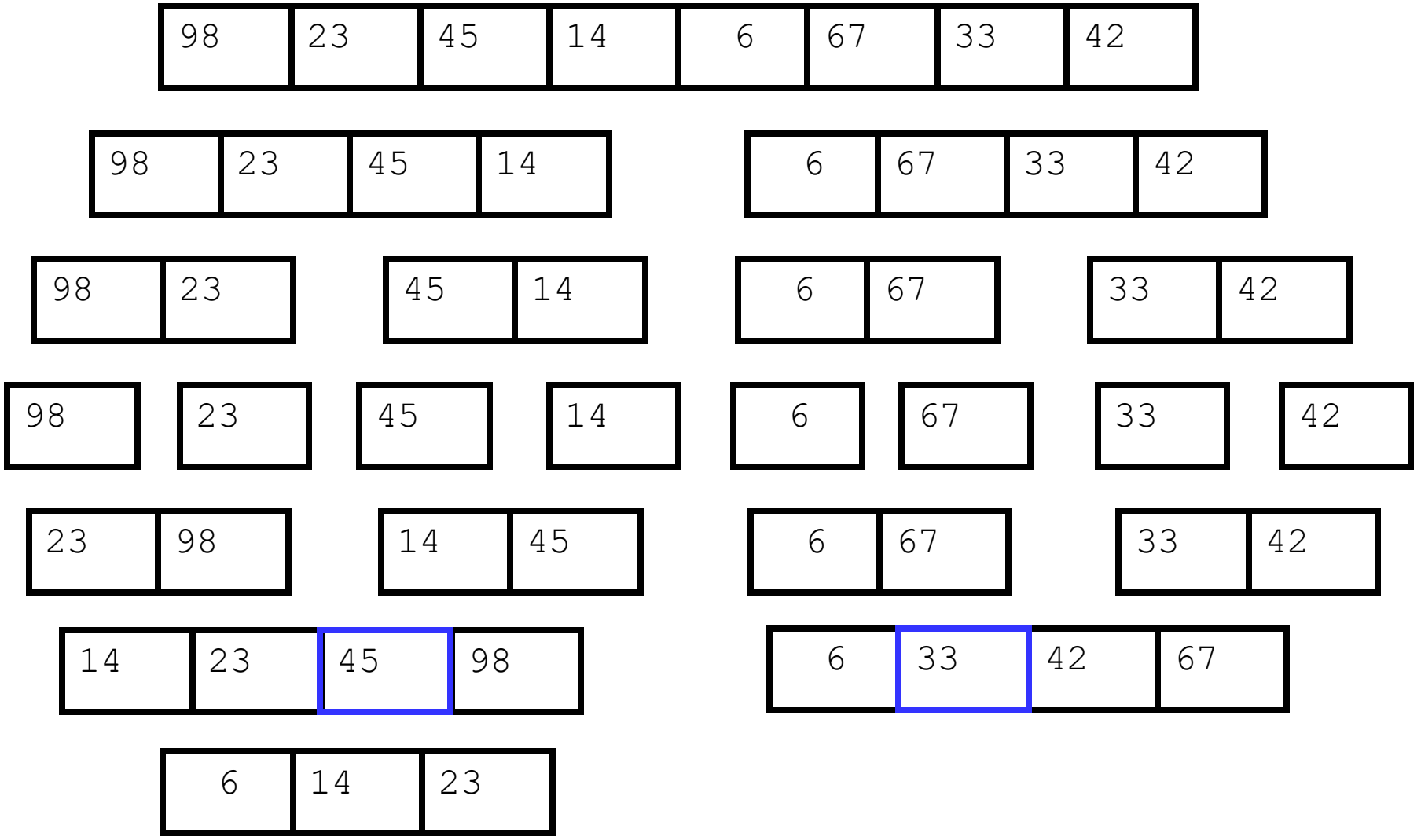

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33
Merge

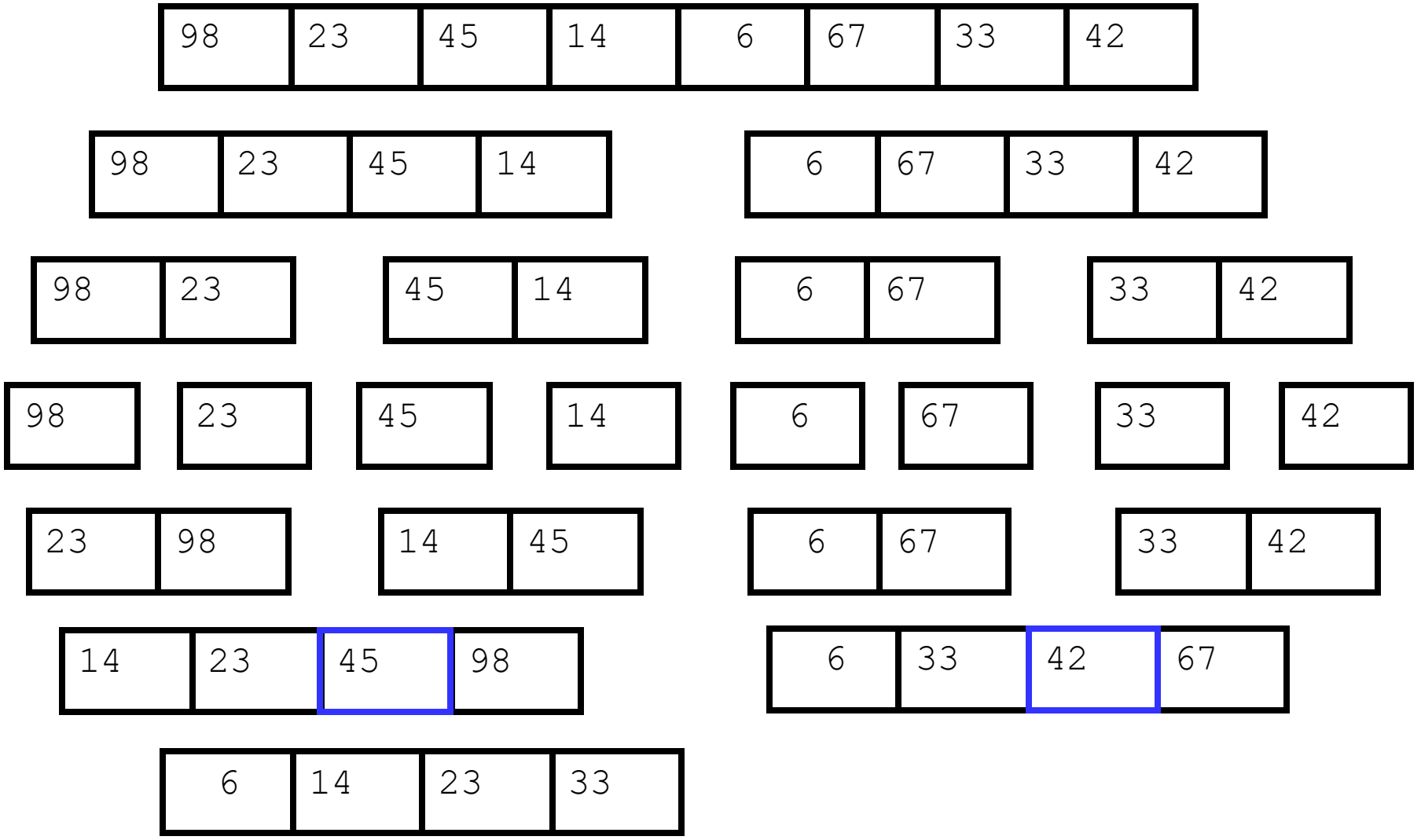

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42
Merge

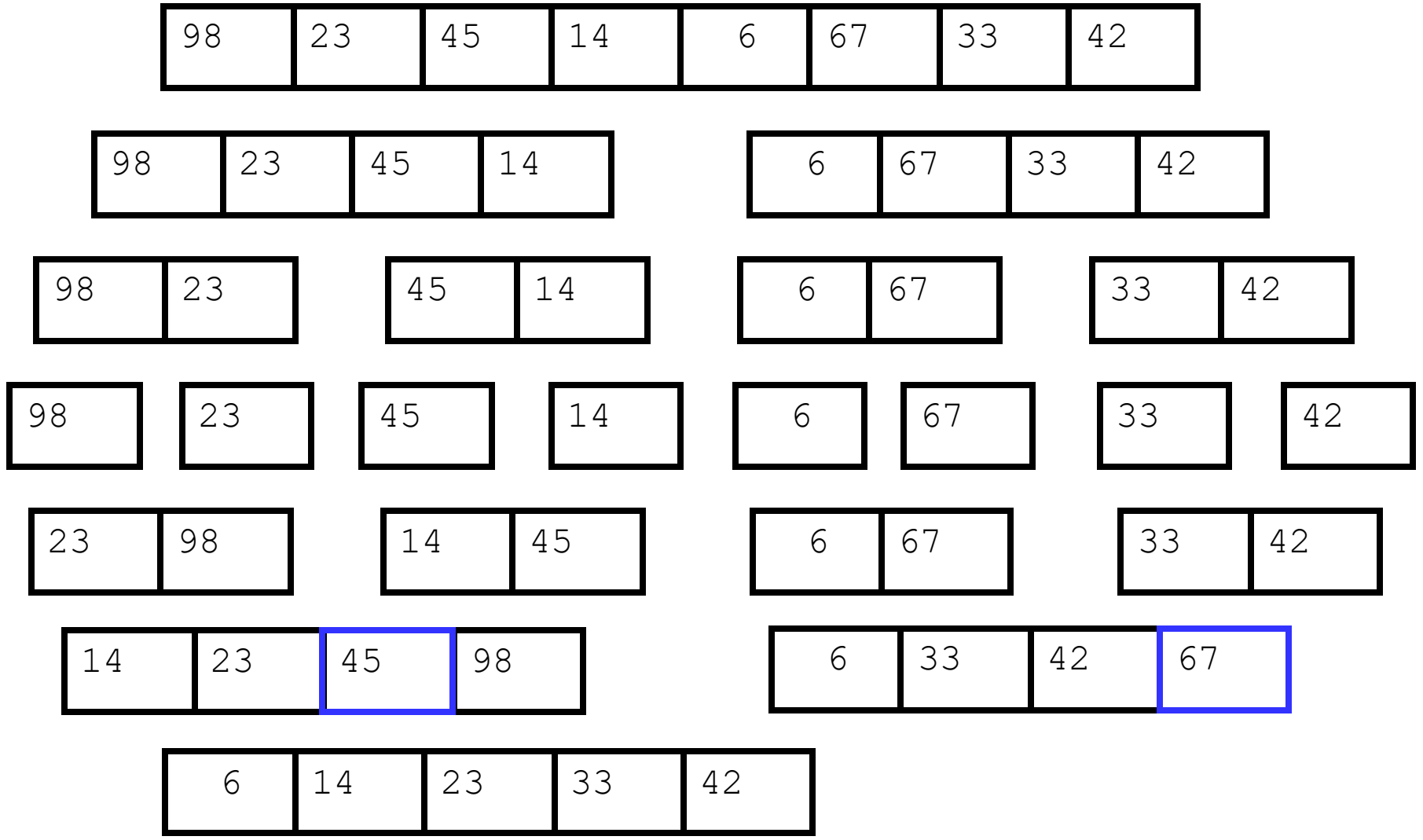

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
Merge

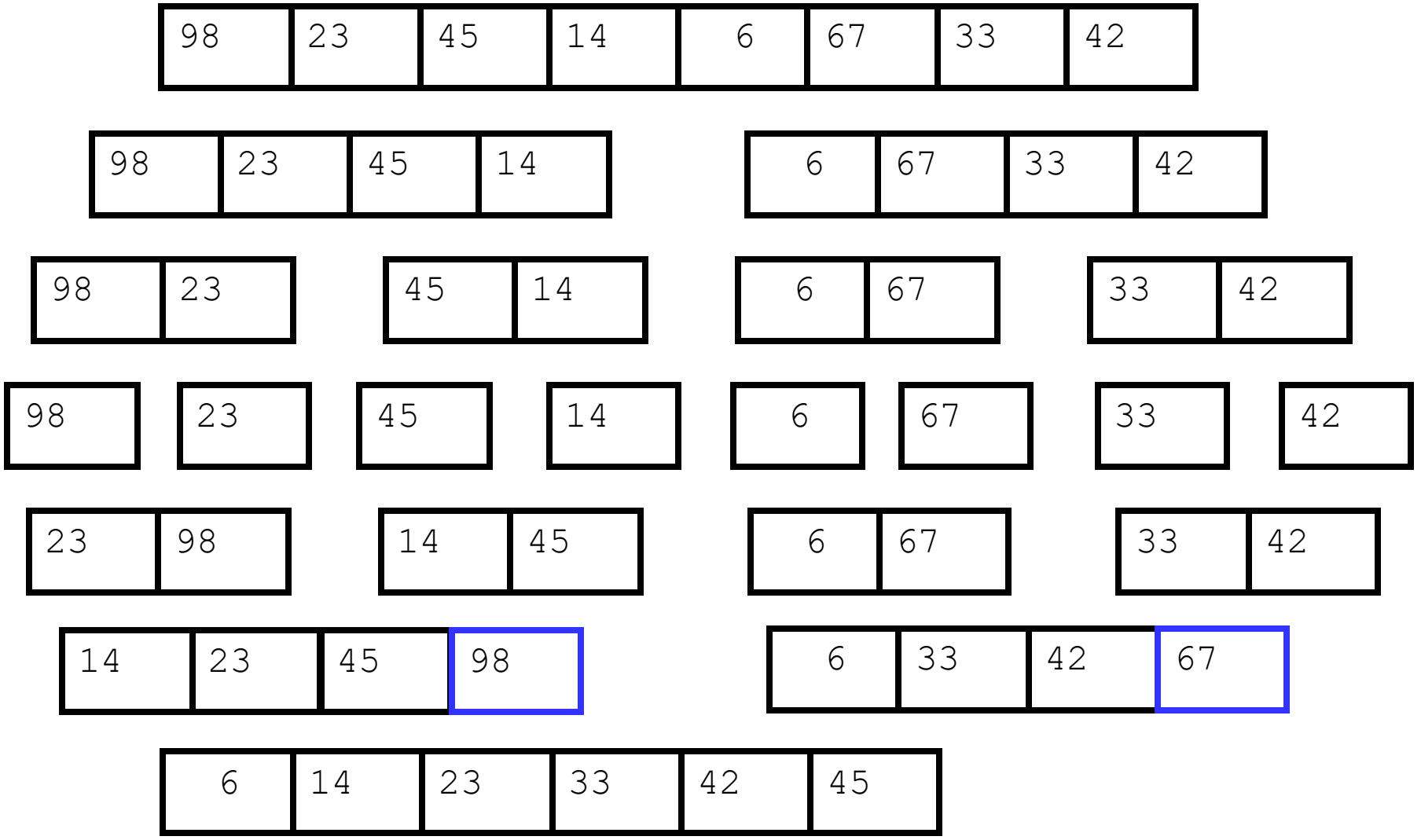

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
Merge

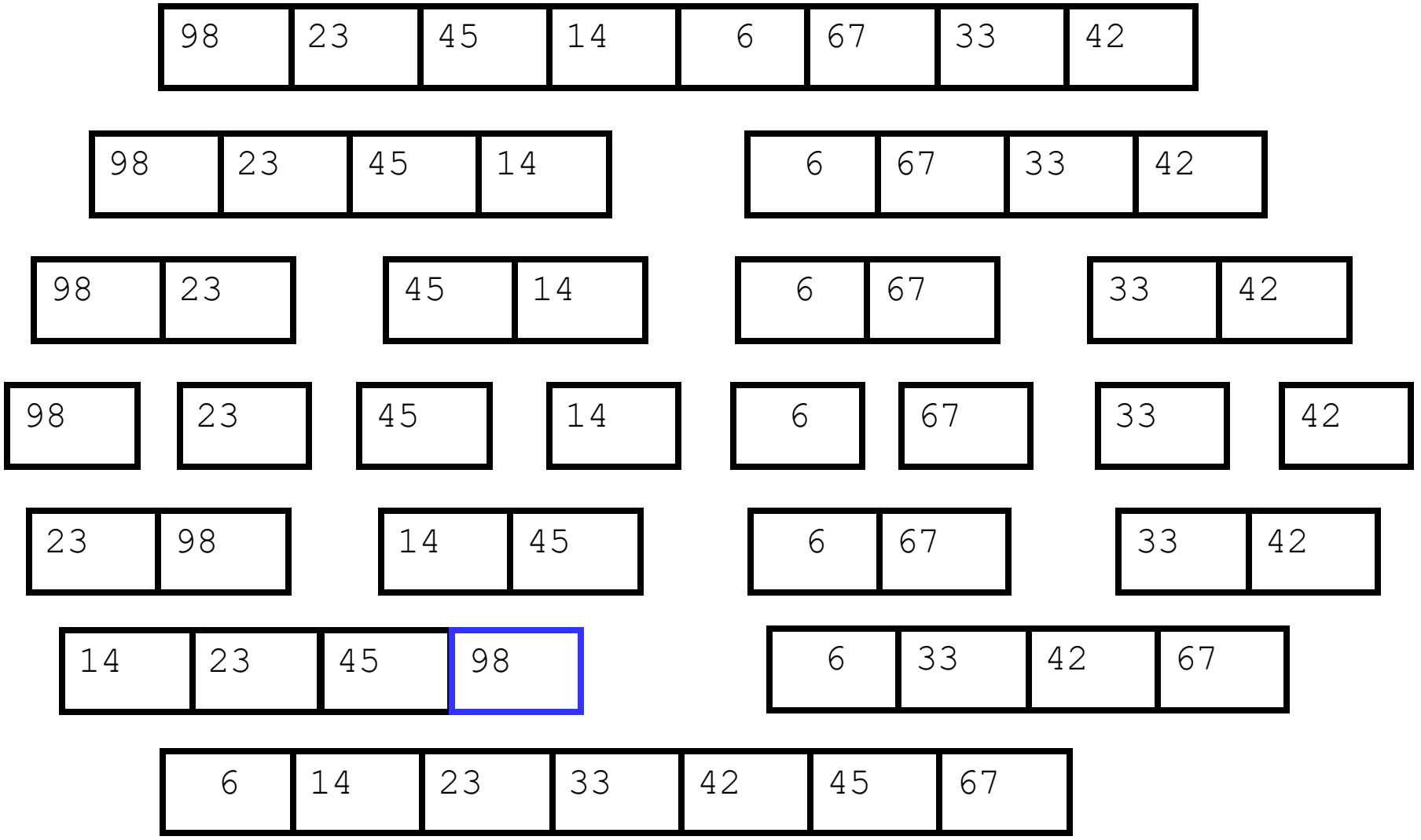

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6
Merge

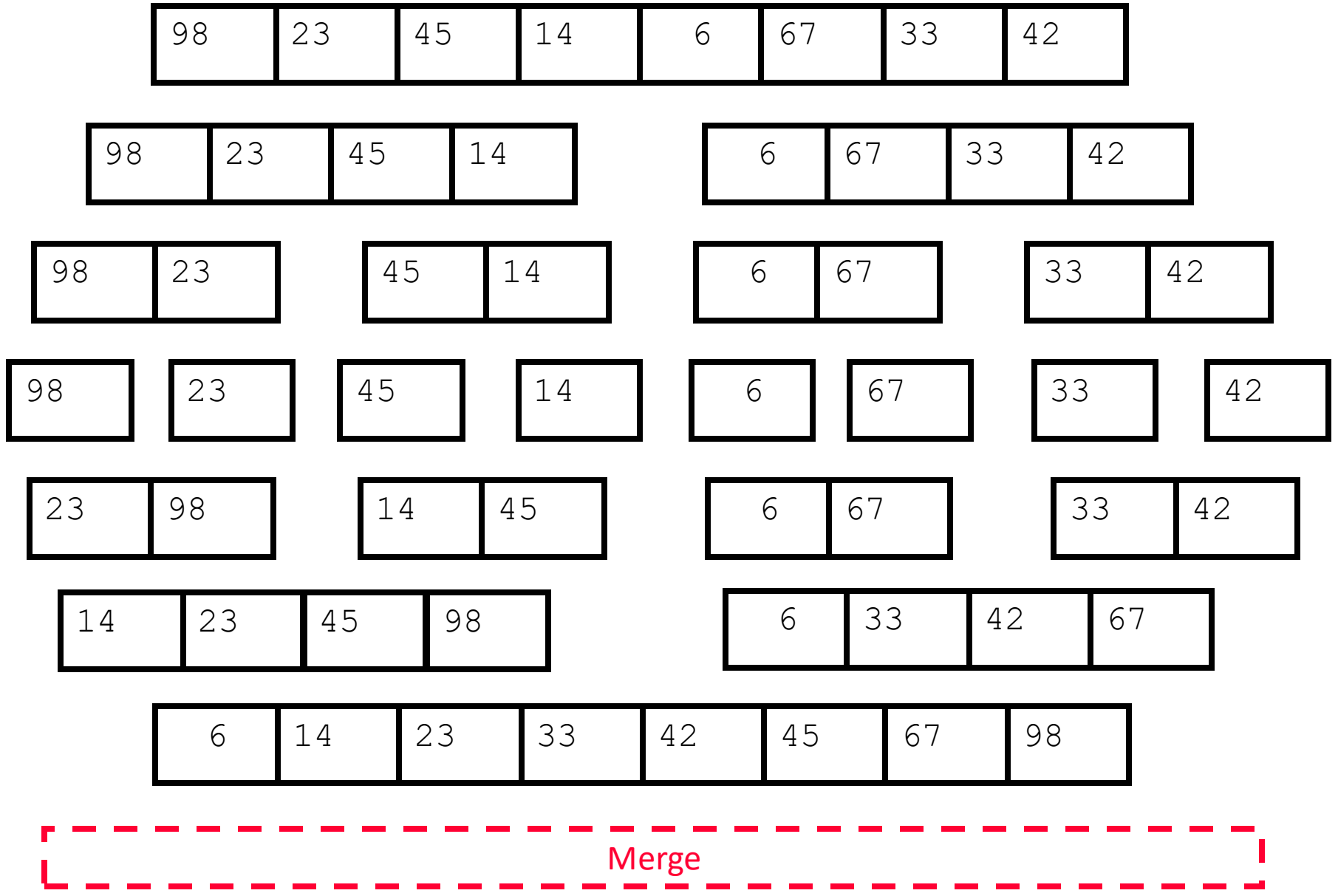

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23
Merge

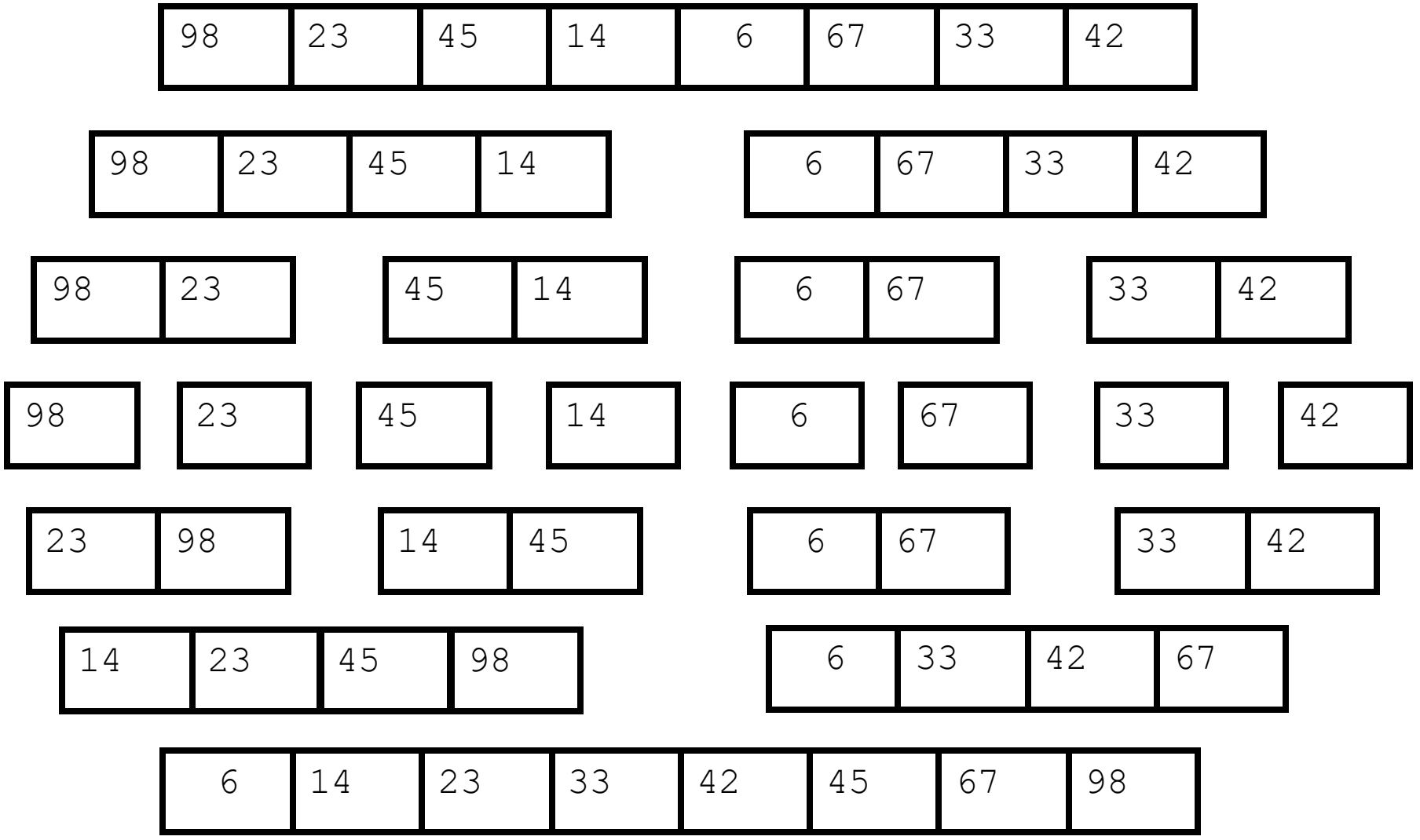

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33
Merge

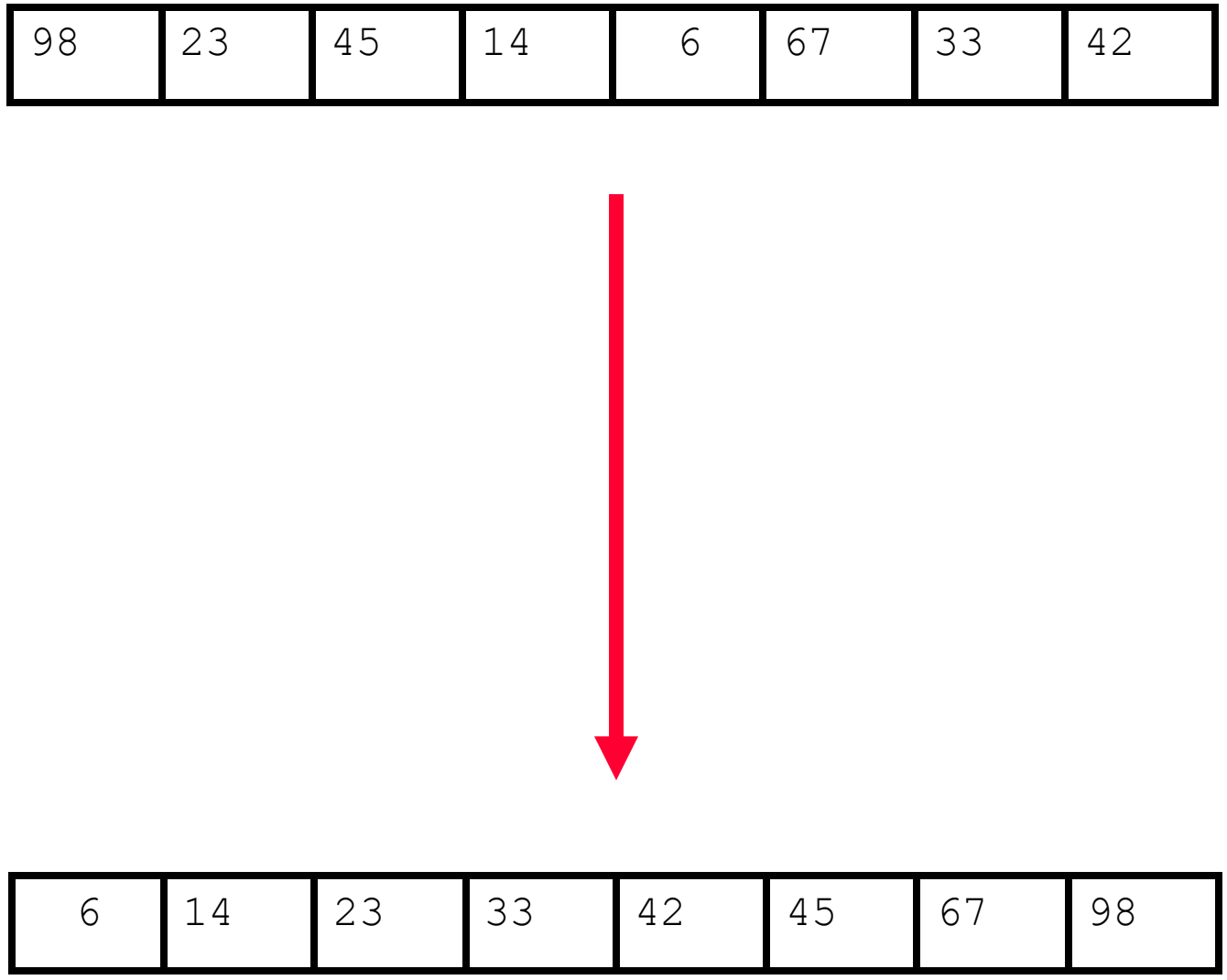

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42
Merge

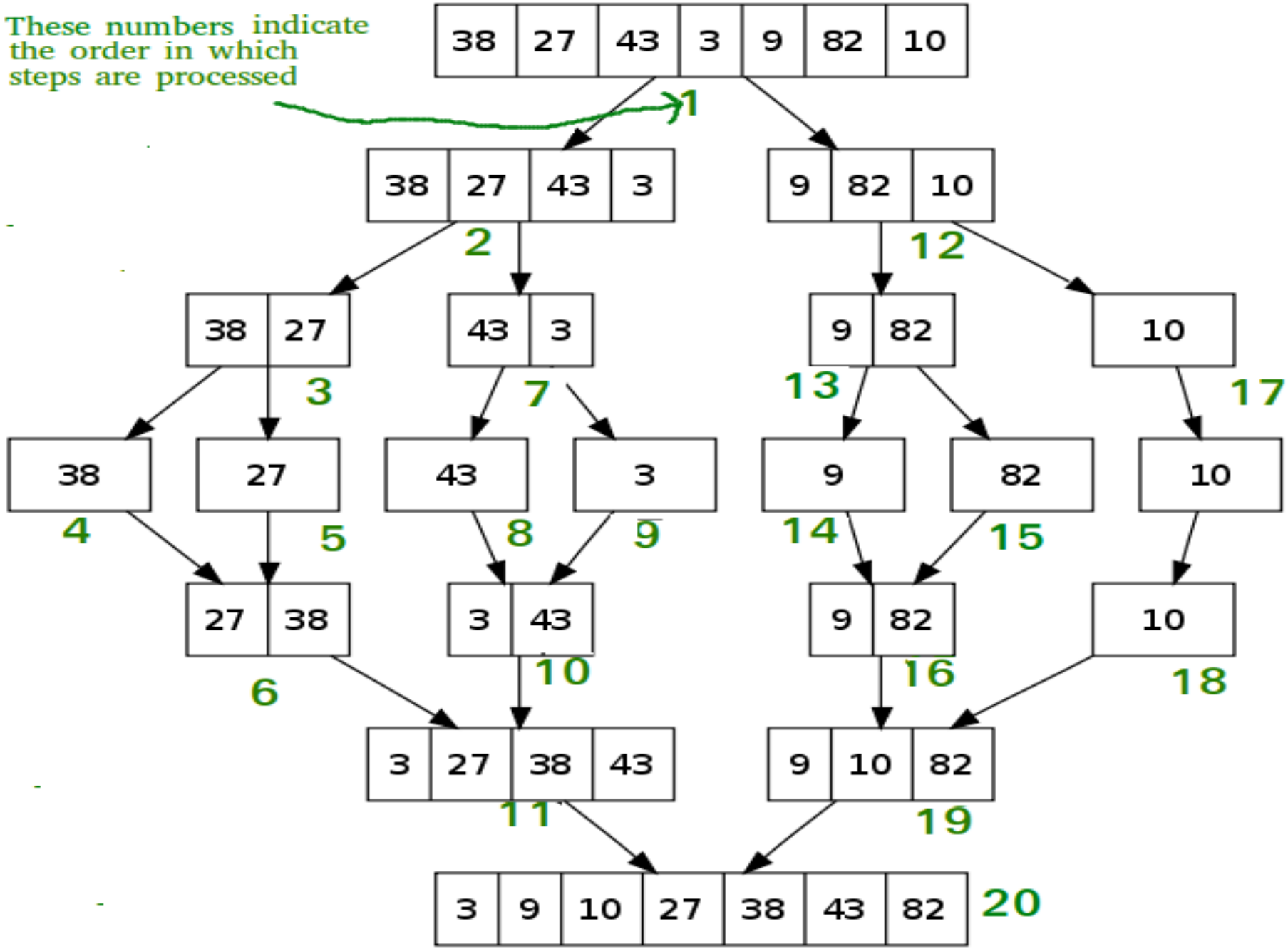

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98


98 23 45 14 6 67 33 42
6 14 23 33 42 45 67 98


These numbers indicate the order in which steps are processed
38 27 43 3 9 82 10
1
38 27 43 3
9 82 10
2
12
38 27
43 3
9 82
10
3
7
13
17
38
27
43
3
9
82
10
4
5
8
9
14
15
27 38
3 43
9 82
10
6
10
16
18
3 27 38 43
9 10 82
11
19
3 9 10 27 38 43 82
20

Merge sort algorithm

```
Algorithm MergeSort (low, high)
 {
 if (low < high) then//if there are more than one element
 {
          mid :=(low + high)/2      //Divide the list into two parts
         MergeSort (low, mid)  // Apply mergesort to the left list
         MergeSort (mid+1, high)// Apply mergesort to the Right list
         Merge (low, mid, high)// Merge two lists into one list
  }
 }
```

```
Algorithm  Merge (low, mid, high)
   {
      h := low, i := low, j := mid + 1
          while ((h ≤ mid) and (j ≤ high)) do
          {
               if (a[h] ≤ a[j]) then
              {    b [i] := a[h]  ;h :=h + 1 }
              else {  b[i] :=a[j],    j := j +1}
           i := i + 1
          }
      if (h > mid) then   //add elements left in the second list
          for k:=j to high  do
          {   b[i]:= a[k];  i:=i+1
          }
      else
      for k:=h to mid do //add elements left in the first list
          { b[i]:= a[k];
            i:=i+1
          }
      for k := low to high do // copying back the sorted list to a[]
      a[k] := b[k]
}
```

Analysis of Merge sort

Time complexity of Mergesort can be described by the recurrence relation :

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

$T(n) = 2(2T(n/4) + n/2) + n$

$= 4\,T(n/4) + 2n$

$= 4(2T(n/8) + n/4) + 2\,n$

$= 8T(n/8) + 3n$

……

$= 2^k\,T(n/2^k) + k\,n$

$= 2^k\,T(1) + k\,n$

$= n + n \log n$

$T(n) = n + n(\log n) = O(n \log n)$

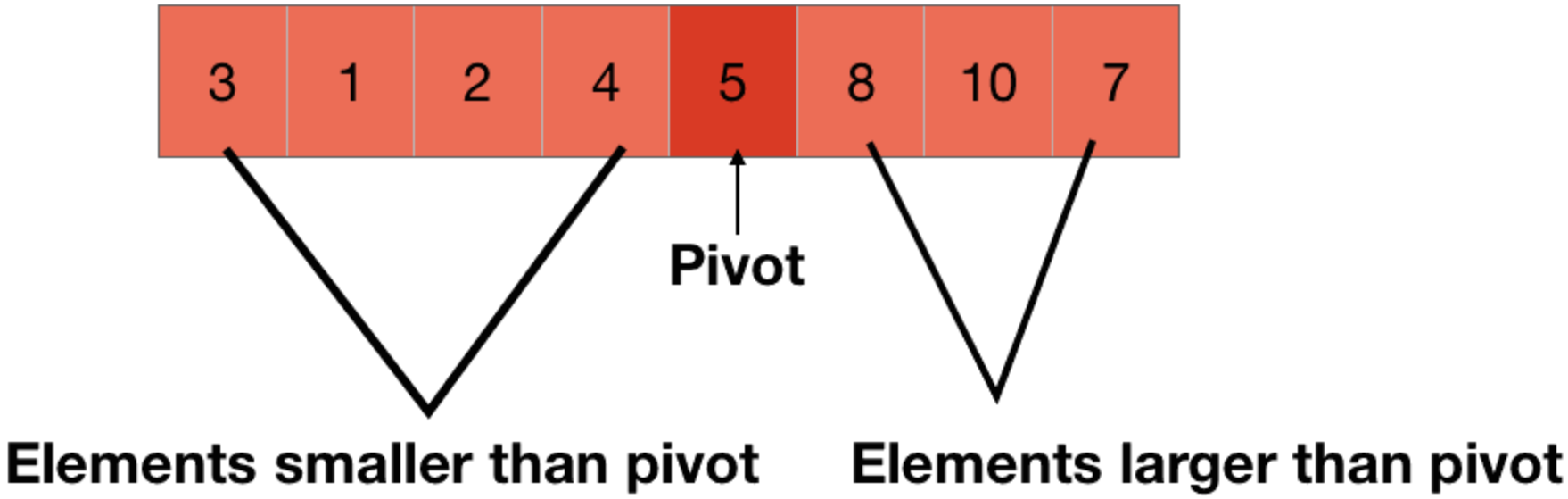
Analysis of Merge sort

- Time complexity of Merge Sort is O(n*log n) in all the 3 cases (worst, average and best) as merge sort always **divides** the array in two halves and takes linear time to **merge** two halves.

Quick sort

- Quick sort is another sorting algorithm which uses Divide and Conquer for its implementation.
- Unlike the Merge Sort, Quick sort doesn't use any extra array in its sorting process.
- Quick sort first chooses a **pivot** and then partition the array around this pivot.
- In the partitioning process, all the elements smaller than the pivot are put on one side of the pivot and all the elements larger than it on the other side.
- This partitioning process is repeated on the smaller sub arrays and hence finally results in a sorted array.

Quick sort


3
1
2
4
5
8
10
7
Pivot
Elements smaller than pivot
Elements larger than pivot

Quick sort

- We can pick Pivot element in different ways:

1. pick first element as pivot.
2. pick last element as pivot
3. Pick a random element as pivot.
4. Pick median as pivot.

Quick sort

- **Quick sort has three steps:**

1. Pick an arbitrary element of the array (the pivot).
2. Divide the array into two segments, those that are smaller and those that are greater, with the pivot in between (partition).
3. Recursively sort the segments to the left and right of the pivot.

Quick sort algorithm

```
Algorithm  Quicksort (a, low, high)
{
if (low < high) then //if there are more than one element
{
 j := partition(a, low, high);
// j is the position of the partitioning element
 Quicksort (a,low, j – 1);
 Quicksort(a,j + 1 , high);
//there is no need for combining solutions
}
}
```

```
int partition(int a[], low , high)
{
v=a[low];i=low; j=high+1;a[j]=∞;
do
{          do{
            i++;
            }while(a[i]<v);
            do{
            j--;
            }while(v<a[j]);
            if(i<j) {swap(a[i[,a[j]);}
   }while(i<j);
a[low]=a[j];a[j]=v;
return(j);}
```

Sort the following list of elements using Quick sort:

10 16 8 12 15 6 3 9 5

45 26 77 14 68 61 97 39 99 90

Analysis of Quick sort

- Let $T(n)$ be the time complexity of Quick sort and if the pivot ends up at position k, then we have

  $T(n) = T(n-k) + T(k) + n$

  $T(1)=1$

- Partition can be done in n comparisons , where n is the size of the array.
- To determine best, worst, and average case complexity we need to determine the values of k that correspond to these cases.

Analysis of Quick sort

- The best case of quicksort occurs when the pivot element divides the array into two exactly equal parts, in every step. The recurrence equation is as follows:

  $T(n) = 2T(n/2) + n$

  $T(1)=1$

  using the substitution method the equation can be written as follows:

  $T(n)=2^kT(n/2^k) + kn$

  so $T(n)=O(nlogn)$

Analysis of Quick sort

- The worst case of quicksort occurs when the first element is always the pivot element .

  The recurrence equation is as follows:

  $T(n) = T(n - 1) + n$

  using the substitution method the equation can be written as follows:

  $T(n)= T(n - k) + (n+(n-1)+(n-2) ..... + (n-k+1))$

  so $T(n)=O(n^2)$

Analysis of Quick sort

- The average case time complexity of quick sort is :
  $T(n)=O(n \log n)$

Quick sort Vs Merge sort

	Best	Worst	Average
✓ merge sort	O(nlogn)	O(nlogn)	O(nlogn)
✓ Quick sort	O(nlogn)	$O(n^2)$	O(nlogn)